Digital Assignment 3 — Review 3

CloudShield

Intelligent Cloud Cost & Security Optimization Platform with a configurable Fake AWS simulator

Item	Value
Submission	Single PDF: integration / regression / mutation testing, version management and system building evidence, screenshots of the developed functionality, tools and technologies
GitHub repository	https://github.com/Nayonika-Nair/CloudShield
Branch evidenced	develop (integration branch), feature branches merged with --no-ff; 36 commits on develop
Generated on	2026-10-16
Evidence policy	Every screenshot was taken from the running application or from real command output captured in this repository (docs/testing/results/*.txt). Nothing is mocked. Limitations are stated where they apply.

How to find each required item

Required by the assignment	Section
Integration / regression / mutation testing (screenshots)	2, 3 and 4
Version management and system building (screenshots)	5 and 6
Screenshots of developed functionalities	7
Tools / technologies used	8
Demo of the project	7.1 (scripted walkthrough) and README demo script
Project report (problem statement, user stories, architecture, tests, appendix: SRS, DFD, ERD, UML, code / GitHub)	Separate PDF: CloudShield_Project_Report.pdf

Contents

1. Project overview **3**

2. Integration testing evidence **5**

3. Regression testing evidence **6**

4. Mutation testing evidence **7**

5. Version management evidence **8**

6. System building evidence **9**

6.1 Container set-up (written; image build not executed) 9

6.2 Native clean start, health check and persistence 9

6.3 Test execution on PostgreSQL 10

7. Screenshots of the developed functionality **10**

7.1 Demonstration walkthrough 10

8. Tools and technologies used **41**

9. Test execution summary **41**

Defects found and fixed while testing 43

10. Conclusion **43**

Appendix A. Evidence register **44**

1. Project overview

CloudShield helps a team understand a cloud estate, find security weaknesses and wasted spend, simulate fixes, apply them after explicit confirmation and verify that the problem is really gone. Because real AWS accounts are not required, the platform runs on a configurable, stateful **Fake AWS simulator**: seeded and reproducible, with VPCs, subnets, route tables, internet gateways, network interfaces, security groups, EC2, EBS, snapshots, Elastic IPs, load balancers, RDS, S3 and IAM, plus a controlled simulated clock, daily metrics and billing.

The core workflow demonstrated in this evidence pack is **Configure** → **Provision** → **Observe** → **Analyze** → **Detect** → **Simulate** → **Remediate** → **Verify** → **Compare** → **Report**. Security findings are *derived from simulated state* (network reachability, IAM policy graph, metrics, billing) by 20 documented rules; scenarios only change state. A finding is resolved only when a re-scan no longer detects the condition.

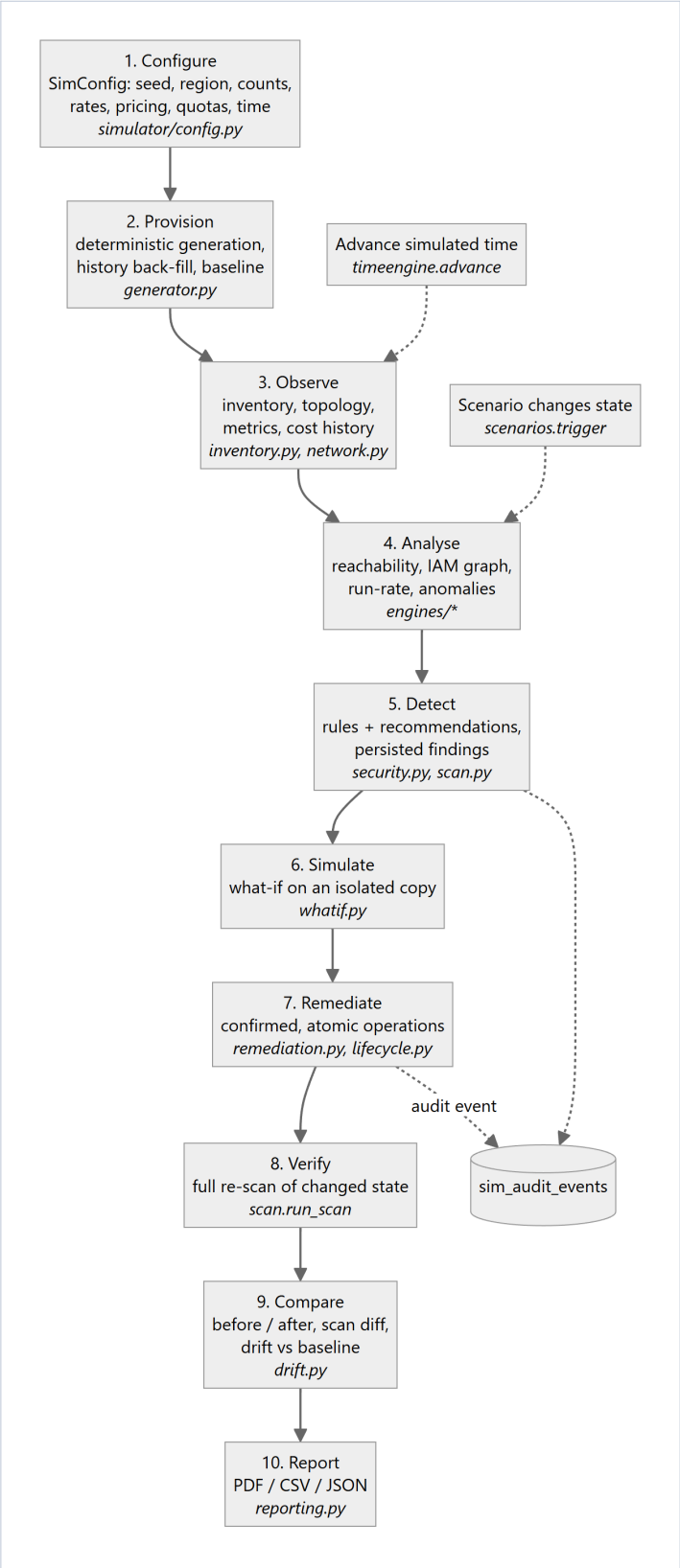


Figure 1. Simulator workflow. The ten stages and the modules that implement them.

2. Integration testing evidence

Integration tests exercise the real database and the real HTTP stack together. They cover simulator ↔ database round trips (resources, relationships, metrics and costs reload to an identical fingerprint), append-only history, optimistic locking against concurrent writers, transactional rollback of a failed mutation (the rejected attempt is still audited), scan reconciliation (idempotent scans, findings that resolve only when the condition disappears), archived environments keeping their rows, and an end-to-end API workflow that follows the demo script. **13 tests ran: 13 passed.** API-level integration is covered by a further 35 tests (Section 9).



Figure 2. Integration tests: simulator <-> database, engines <-> persisted history. Integration-testing evidence (simulator and database, optimistic locking, transactions, scan reconciliation).

Integration evidence also includes the same suite against real PostgreSQL 16 (Section 9) and the frontend contract tests: every API call made by the browser code and every `data-action` hook is checked against the backend's OpenAPI document and handler registry.

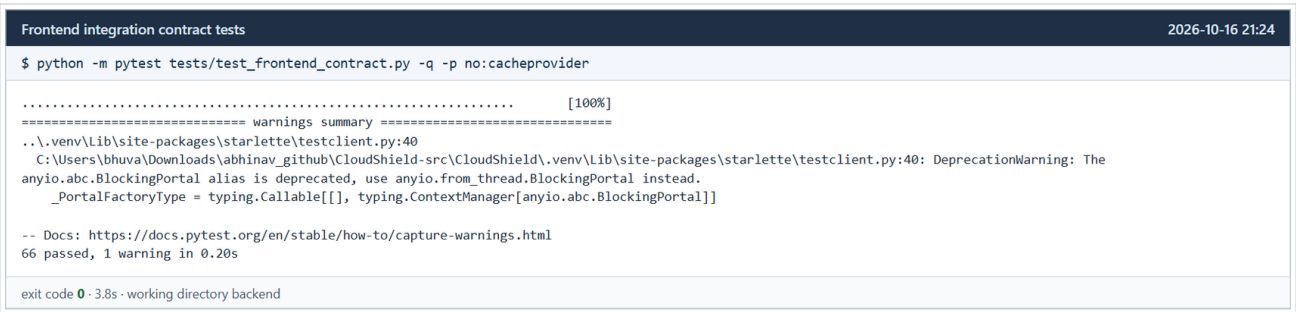


Figure 3. Frontend integration contract tests. Every API call made by the UI exists in the backend OpenAPI document; every button/handler is wired.

3. Regression testing evidence

The original CloudShield code (registration/login, the mock cloud provider, the label-based security, cost and recommendation engines, the original `/api/*` routes and their persistence) was kept and is pinned by regression tests that assert its previously observable behaviour: the 15-resource mock inventory, the baseline scan (score 96, two findings, $1,120 monthly cost, $192 potential saving), route contracts, transactional persistence, and that a legacy scan never touches simulator history. A database created before role-based access control (no `users.role` column) is upgraded in place at start-up. **25 regression tests ran: 25 passed.**

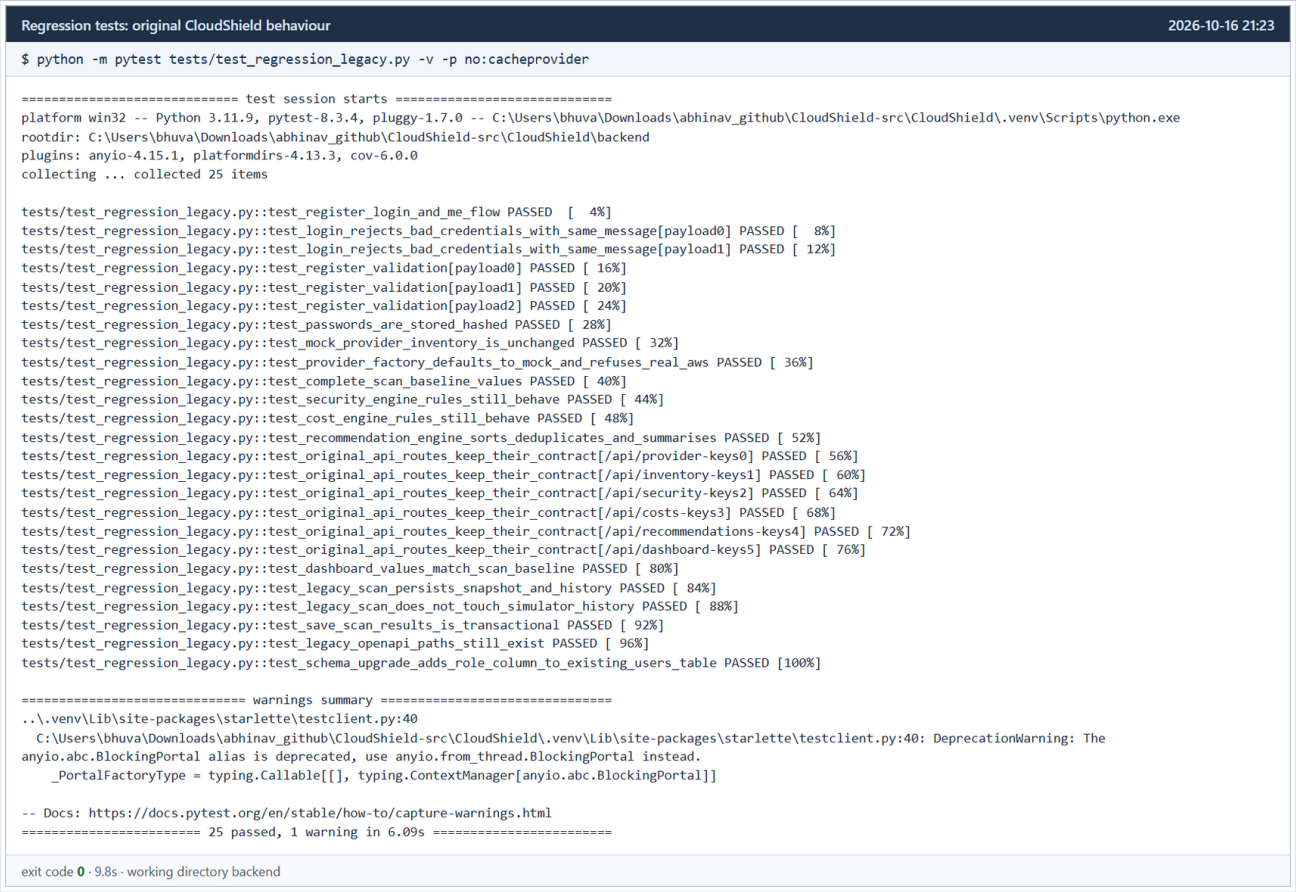


Figure 4. Regression tests: original CloudShield behaviour. Regression evidence: original authentication, mock provider, label-based engines, original API routes and persistence still behave as before.

4. Mutation testing evidence

Tool: **Cosmic Ray 8.4.1** (Python mutation testing). Each target module is mutated in its own copy of the backend so runs cannot contaminate each other. Honest adjustments: (1) a **seeded random sample** of mutants per module was run (seed 2026) because an exhaustive run costs several seconds per mutant on the development machine, and (2) mutants inside **type annotations** were excluded, because they either crash at import (a trivially 'killed' mutant) or have no runtime effect. **Round 1** used the module's targeted pytest files and scored 60.0%. The survivors were then read, boundary and exactness tests were written for them (`test_hardening_*.py`), and **round 2** re-ran only those survivors against the original plus the hardening tests; kills from round 1 were kept unchanged. A kill counts only if the recorded output shows a failing test.

Module	Covers	Gen.	Annot. (excl.)	Tested	Killed	Surv.	Round 1	Final
billing.py	Cost calculation (simulated pricing arithmetic)	323	22	100	54	4	90.0%	93.1%
config.py	Configuration validation logic	690	11	100	78	22	53.0%	78.0%
cost.py	Cost run-rate, history and anomaly detection	402	0	100	79	15	58.5%	84.0%
iam.py	IAM evaluation, trust and escalation	353	22	100	77	23	63.0%	77.0%
network.py	Network reachability analysis	397	44	100	75	25	40.0%	75.0%
optimization.py	Cost recommendation rules	937	66	100	80	8	58.1%	90.9%
remediation.py	Remediation execution and plan hashing	17	0	17	16	0	62.5%	100.0%
security.py	Security detection rules and severity logic	729	66	100	90	10	69.0%	90.0%
All		3848	231	717	549	107	60.0%	83.7%

Table 1. Mutation testing results exactly as recorded by scripts/run_mutation.py (docs/testing/mutation/*.json; round 1 in docs/testing/mutation-round1). Score = killed / (killed + survived); incompetent mutants are excluded.

Surviving mutants. app/simulator/billing.py: 4; app/simulator/config.py: 22; app/engines/cost.py: 15; app/engines/iam.py: 23; app/engines/network.py: 25; app/engines/optimization.py: 8; app/engines/security.py: 10. Their diffs are stored in docs/testing/mutation/<module>.survivors.txt and discussed in the project report.

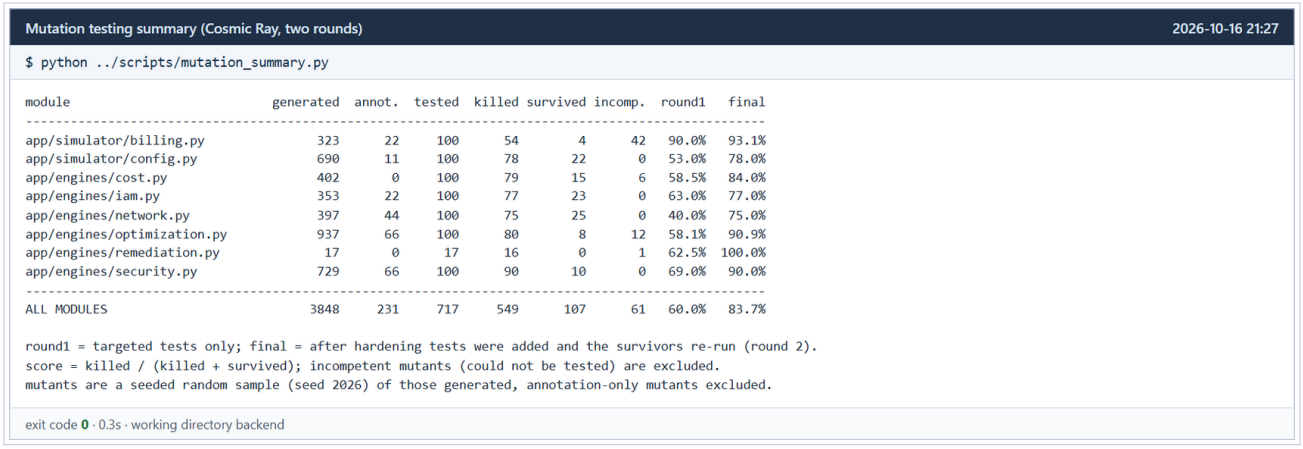


Figure 5. Mutation testing summary (Cosmic Ray, two rounds). Mutation-testing evidence: per-module killed/survived counts, round 1 (targeted tests) and final (after hardening tests).

Interpretation and limits. The score applies only to the sampled mutants of the listed modules, not to the whole code base. A killed mutant means at least one test failed; it does not by itself prove that the failure is meaningful. Lifecycle validation (lifecycle.py) is covered by unit tests but was not mutation-tested because of its size. Some survivors are probably equivalent mutants (a change with no observable behaviour) and cannot be killed.

5. Version management evidence

Repository: `https://github.com/Nayonika-Nair/CloudShield`. Work followed the project's GitHub-Flow with a `develop` integration branch: feature branches (`feature/simulator-core`, `feature/frontend-integration`, `feature/documentation-evidence`) are merged with `--no-ff`, commits are small and describe what changed, and the commits made for this work carry the co-author trailer. The pre-existing branches (`main`, `feature/authentication`) and the user's earlier history were left untouched; the tool never pushes - publishing the branches to GitHub is left to the repository owner.

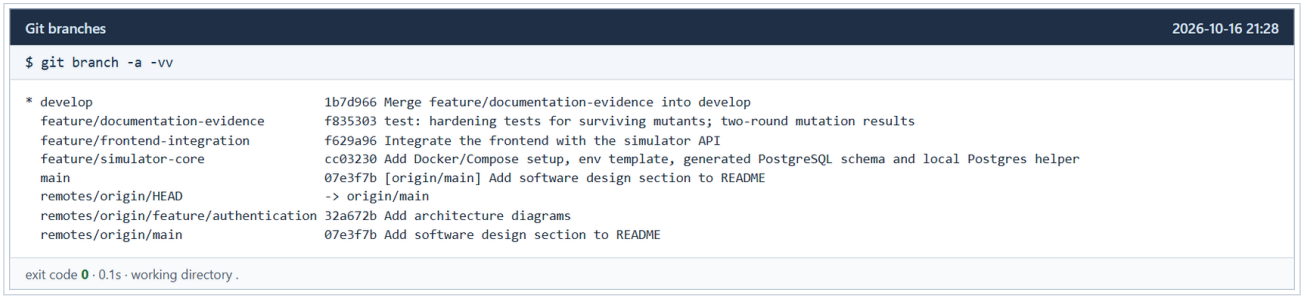


Figure 6. Git branches. Branches used by the GitHub-Flow workflow (main, develop, feature branches).

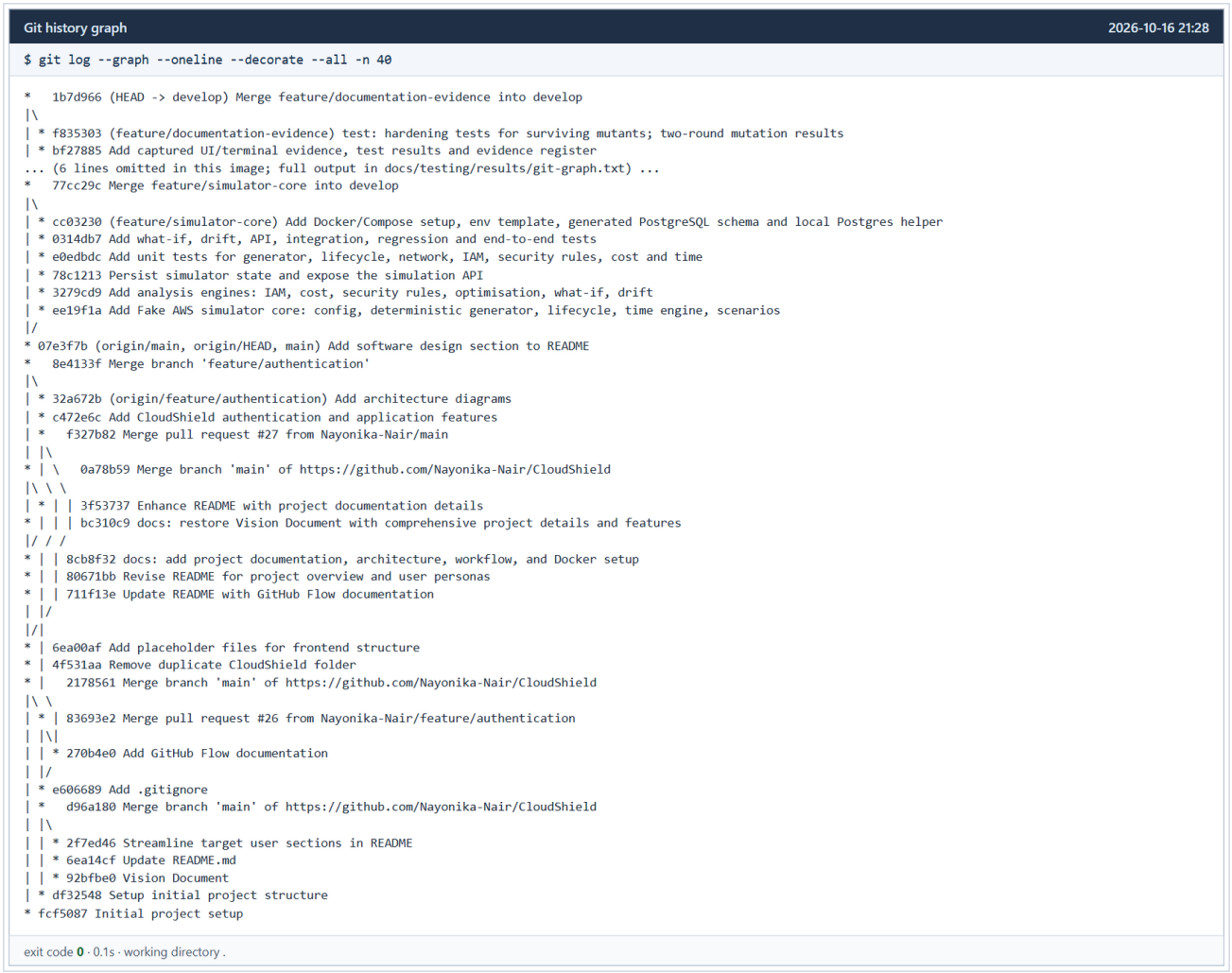


Figure 7. Git history graph. Meaningful commits and --no-ff merges of feature branches into develop.



Figure 8. Repository, remote and working-tree state. Remote URL and clean working tree after the commits (nothing is pushed by the tooling).

6. System building evidence

6.1 Container set-up (written; image build not executed)

The repository provides `docker-compose.yml` (PostgreSQL 16 with a named volume and `pg_isready` health check, the FastAPI backend with a health check and non-root user, and an nginx frontend that serves the static UI and proxies `/api`), two Dockerfiles, `.env.example` and a generated `database/schema.sql`. **Docker is not installed on the machine used for development, so** `docker compose up --build` **was NOT run and no successful container build is claimed.** What was done instead is shown below: a record of the missing tool, static validation of the compose file and Dockerfiles, and a native run of the identical stack.

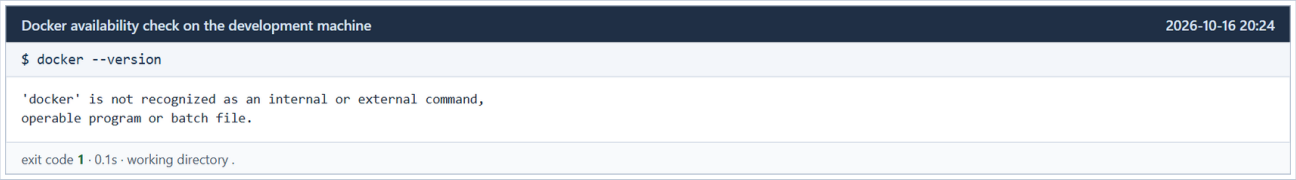


Figure 9. Docker availability check on the development machine. Honest record: Docker is not installed here, so image build / compose up could not be executed.



Figure 10. Static validation of docker-compose.yml and Dockerfiles. Compose file parses, defines db/backend/frontend with health checks, a named volume and dependency conditions (not a substitute for a real build).

6.2 Native clean start, health check and persistence

FastAPI/uvicorn was started against a freshly created, empty PostgreSQL 16 database. Tables were created automatically, the health endpoint reported `database: ok`, an environment was generated and scanned, the server process was stopped and started again, and the clock, fingerprint, scan history and findings were identical afterwards.



Figure 11. Clean start, health check and persistence across restart (native, PostgreSQL 16). The same stack (FastAPI + PostgreSQL) started from an empty database, health-checked, restarted, with identical data afterwards.

6.3 Test execution on PostgreSQL



Figure 12. Full suite on PostgreSQL 16. The same suite executed against a real PostgreSQL 16 server (TEST_DATABASE_URL).

7. Screenshots of the developed functionality

7.1 Demonstration walkthrough

All screenshots in this section were produced by `scripts/ui_walkthrough.py`, which drives the real application in Chrome with Playwright against the FastAPI + PostgreSQL stack, asserts what it sees (for example that a world-open rule exists before remediation and is gone afterwards) and fails if the browser logs any error. The sequence follows the demo script: sign in, configure and generate, inspect, trigger a scenario, scan, explain a finding, preview, apply and verify a remediation, cost anomaly and recommendation, what-if, audit, drift and report.

Authentication and dashboard

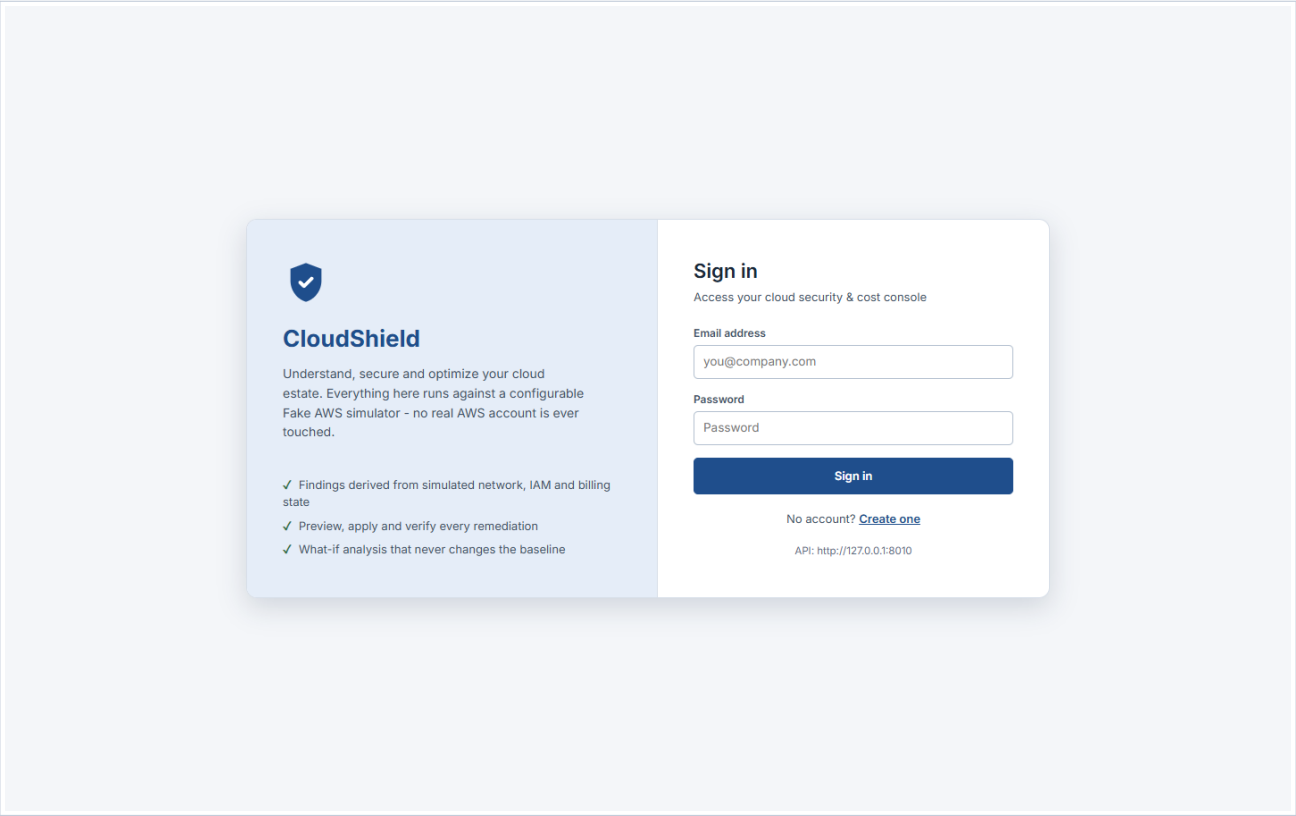


Figure 13. Sign-in screen. Login view; credentials are validated by the API, not hard-coded in the page.

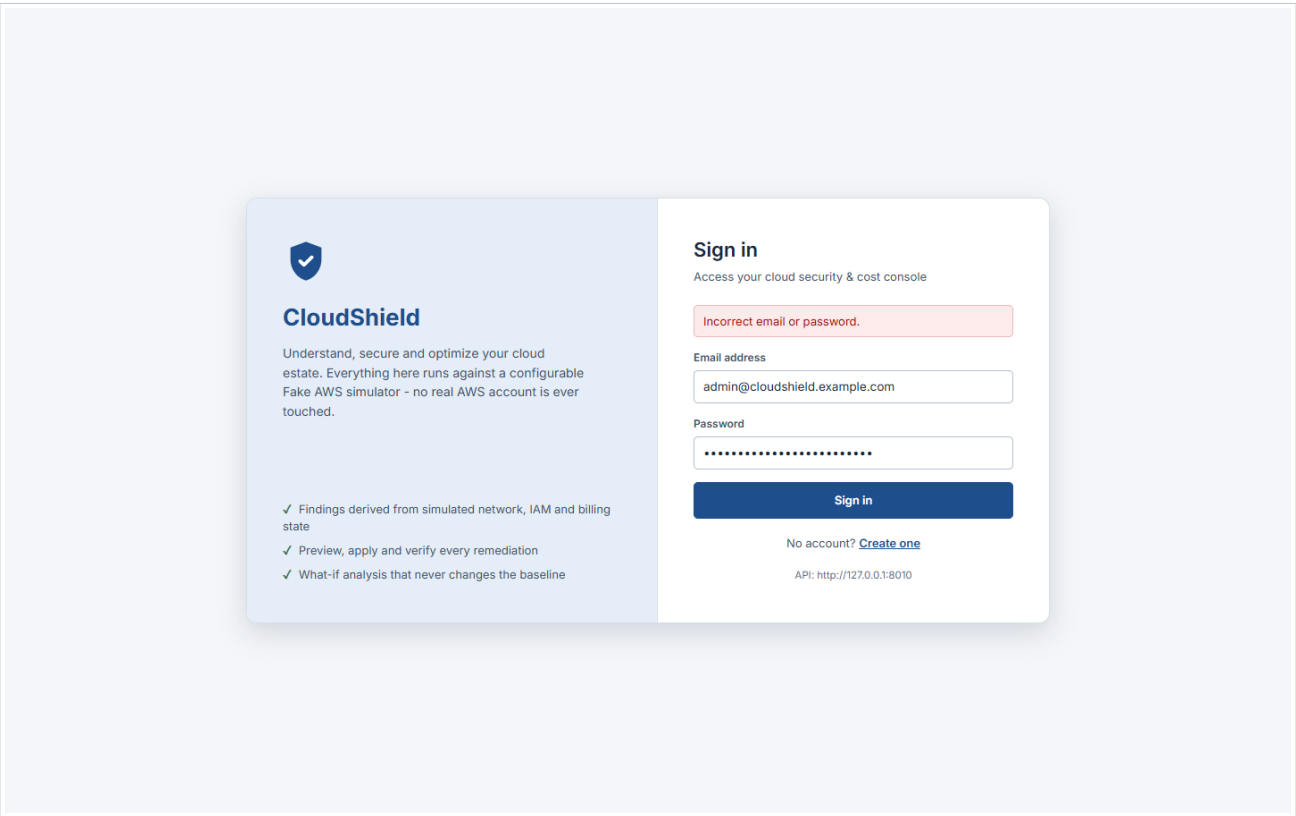


Figure 14. Rejected sign-in. A wrong password is refused with a generic message (HTTP 401).

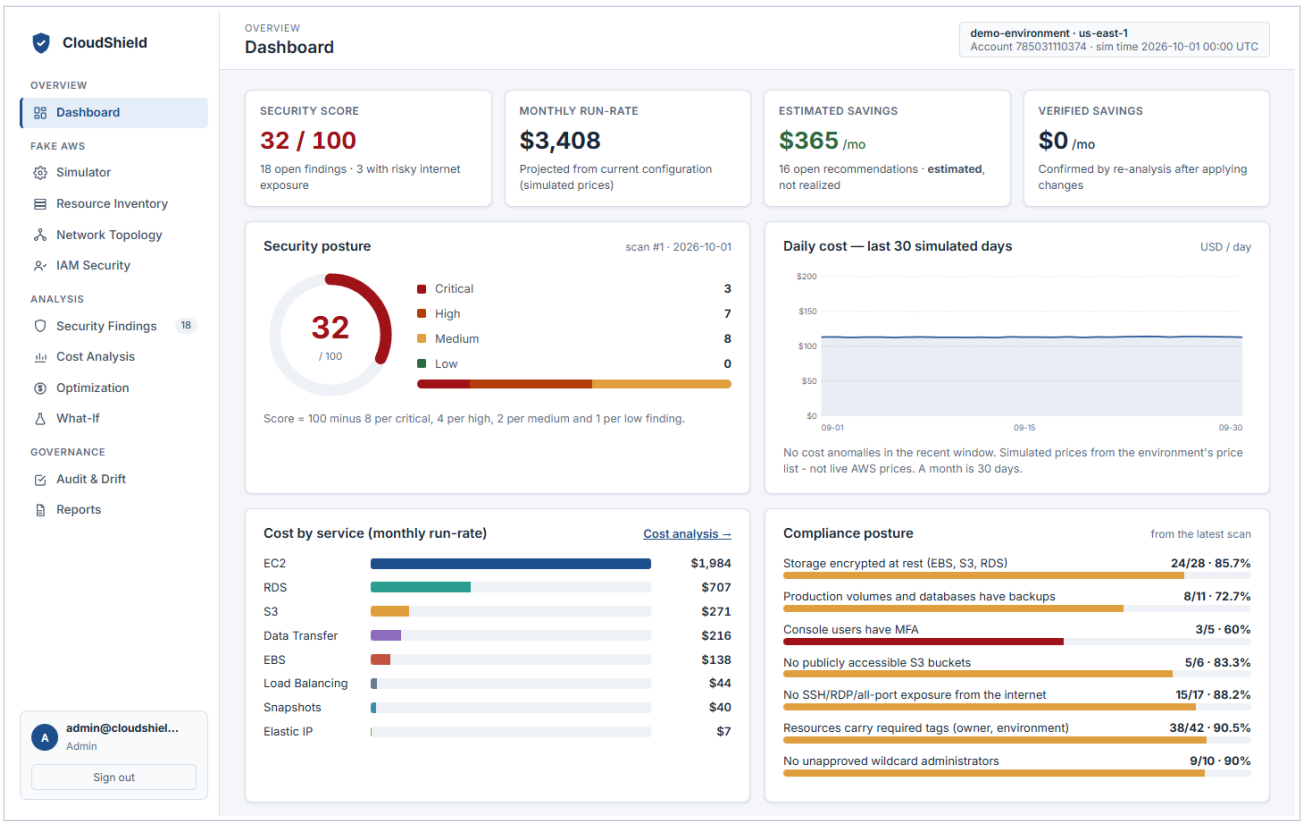


Figure 15. Dashboard (default environment). KPIs, security posture, cost trend, cost by service and compliance derived from the persisted simulated environment.

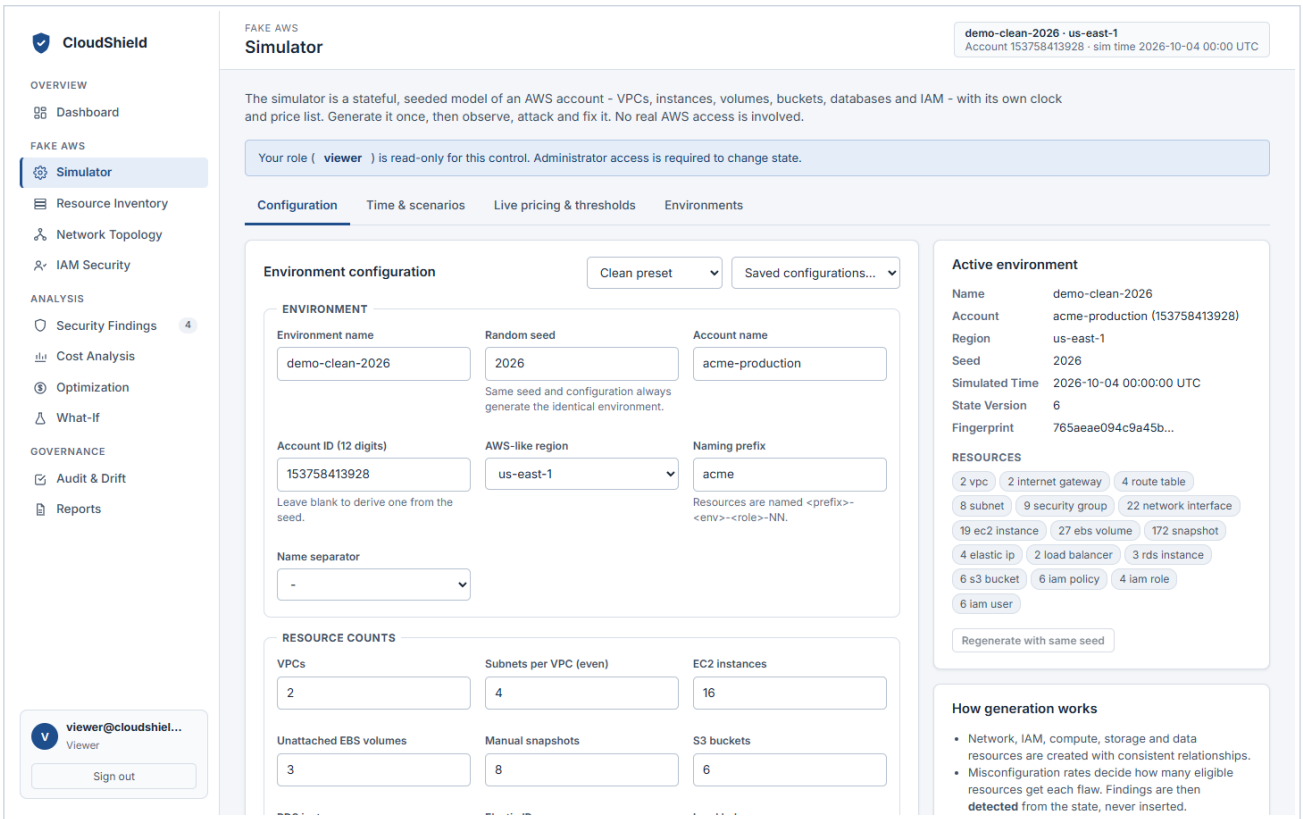


Figure 16. Read-only viewer role. A viewer sees a notice and cannot generate environments or change state (also enforced by the API with HTTP 403).

Simulator configuration and generation

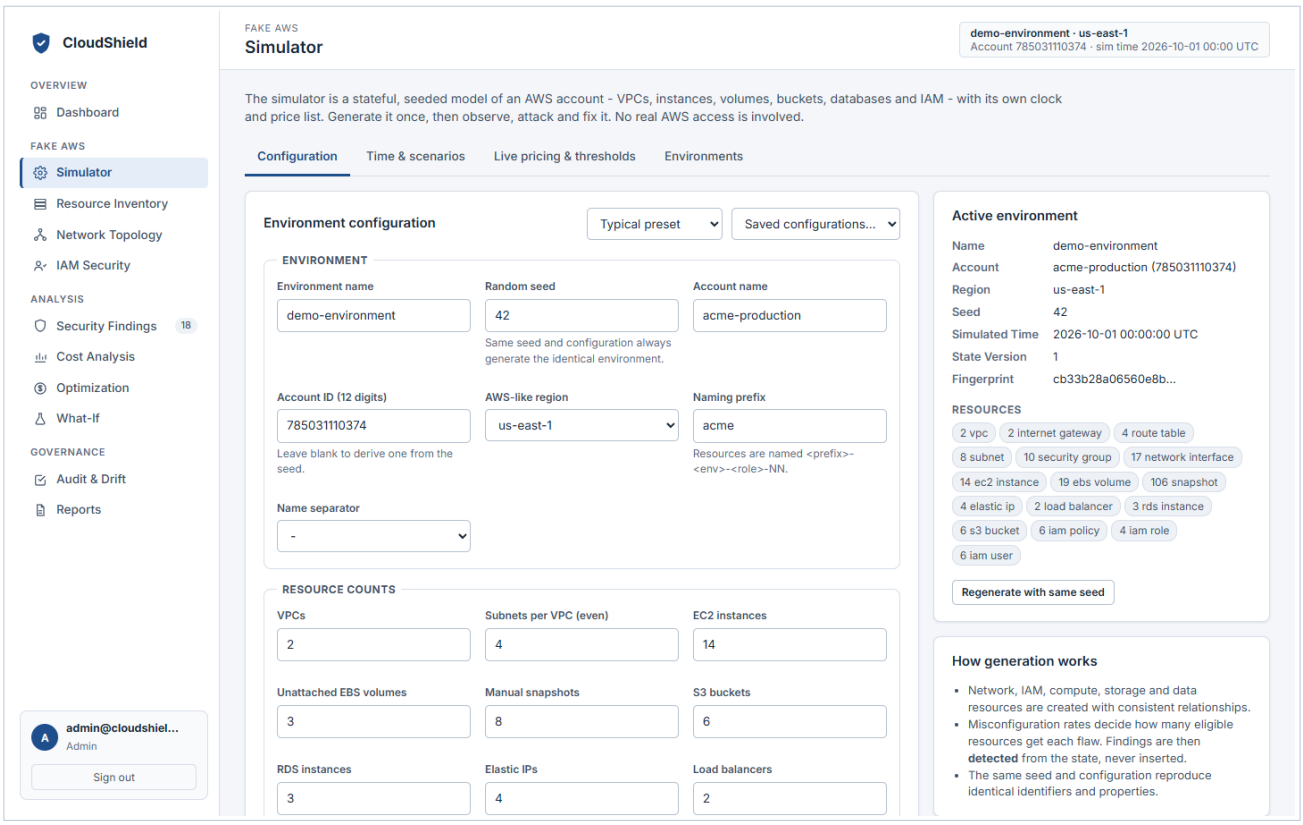


Figure 17. Simulator configuration. Seed, region, resource counts, utilization, misconfiguration rates, quotas, time, pricing and scenario probabilities.

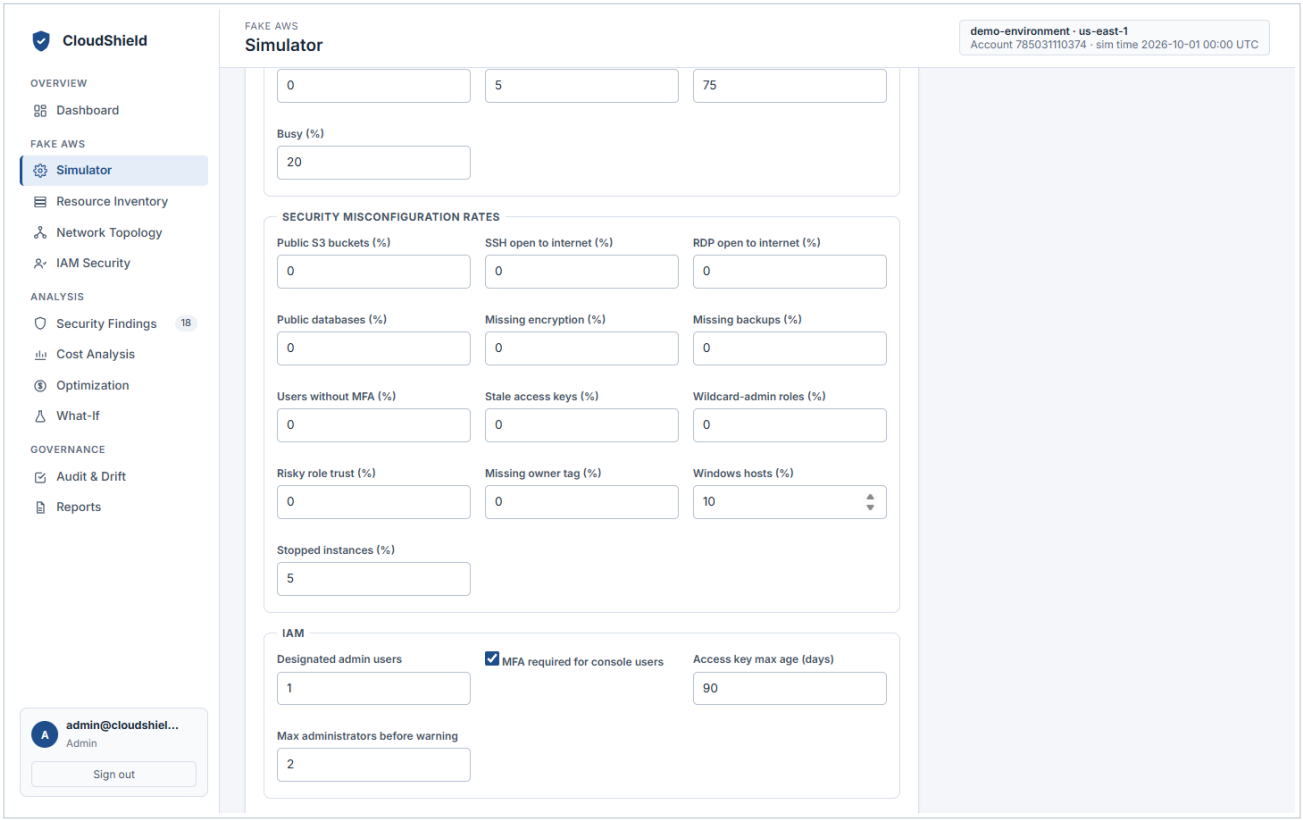


Figure 18. Misconfiguration rates and scenario probabilities. Clean preset: misconfiguration rates are zero, so no flaw exists until a scenario creates it.

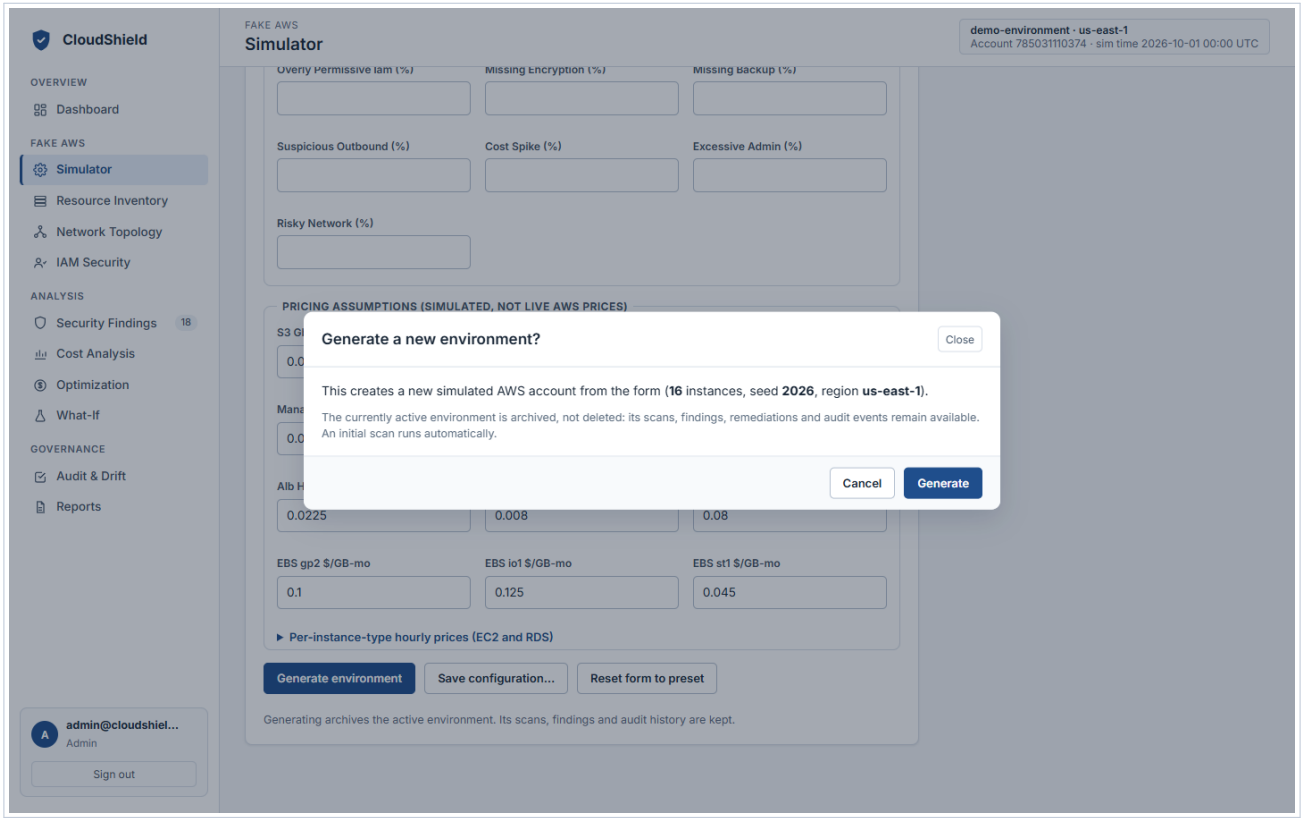


Figure 19. Generate confirmation. Generation archives (does not delete) the previous environment and its history.

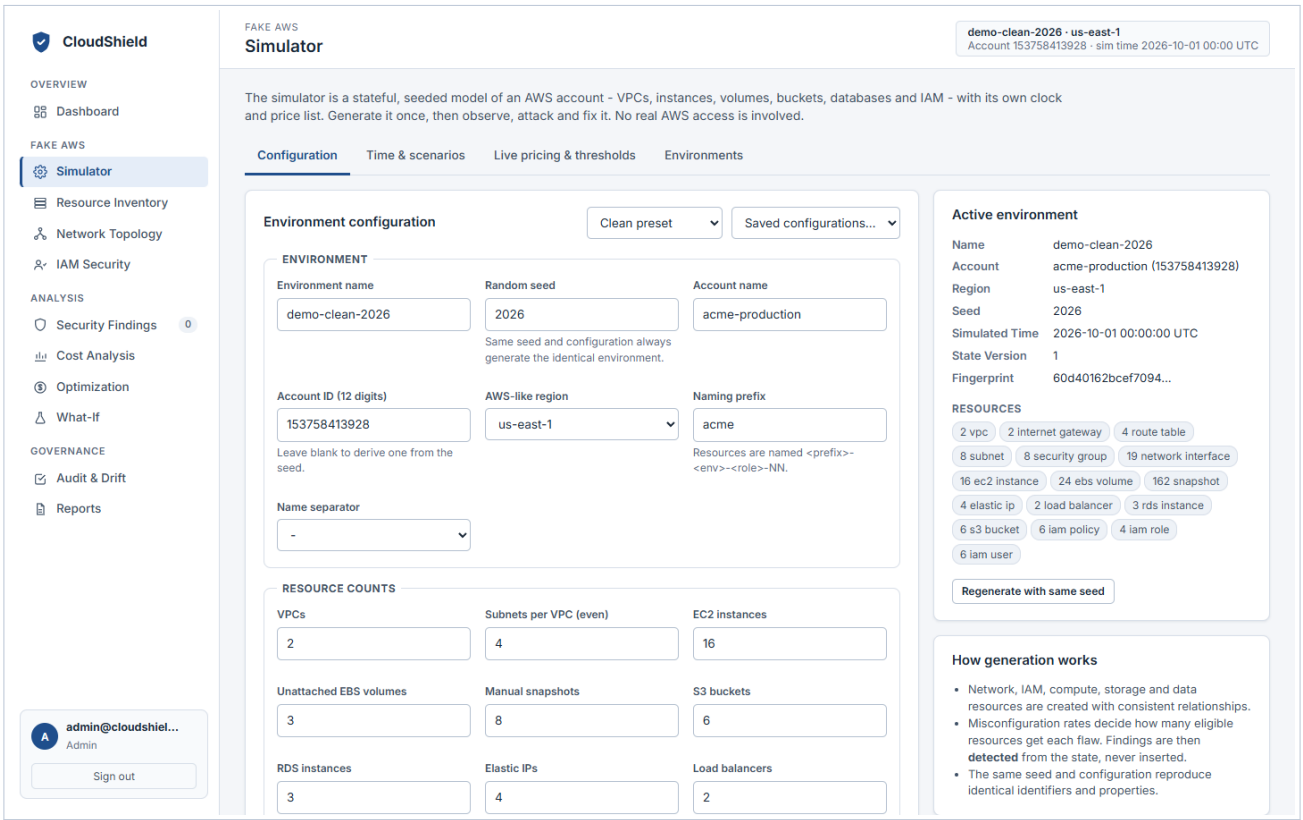


Figure 20. Environment generated. Active environment panel shows the seed, simulated clock, fingerprint and resulting resource counts (16 EC2 instances).

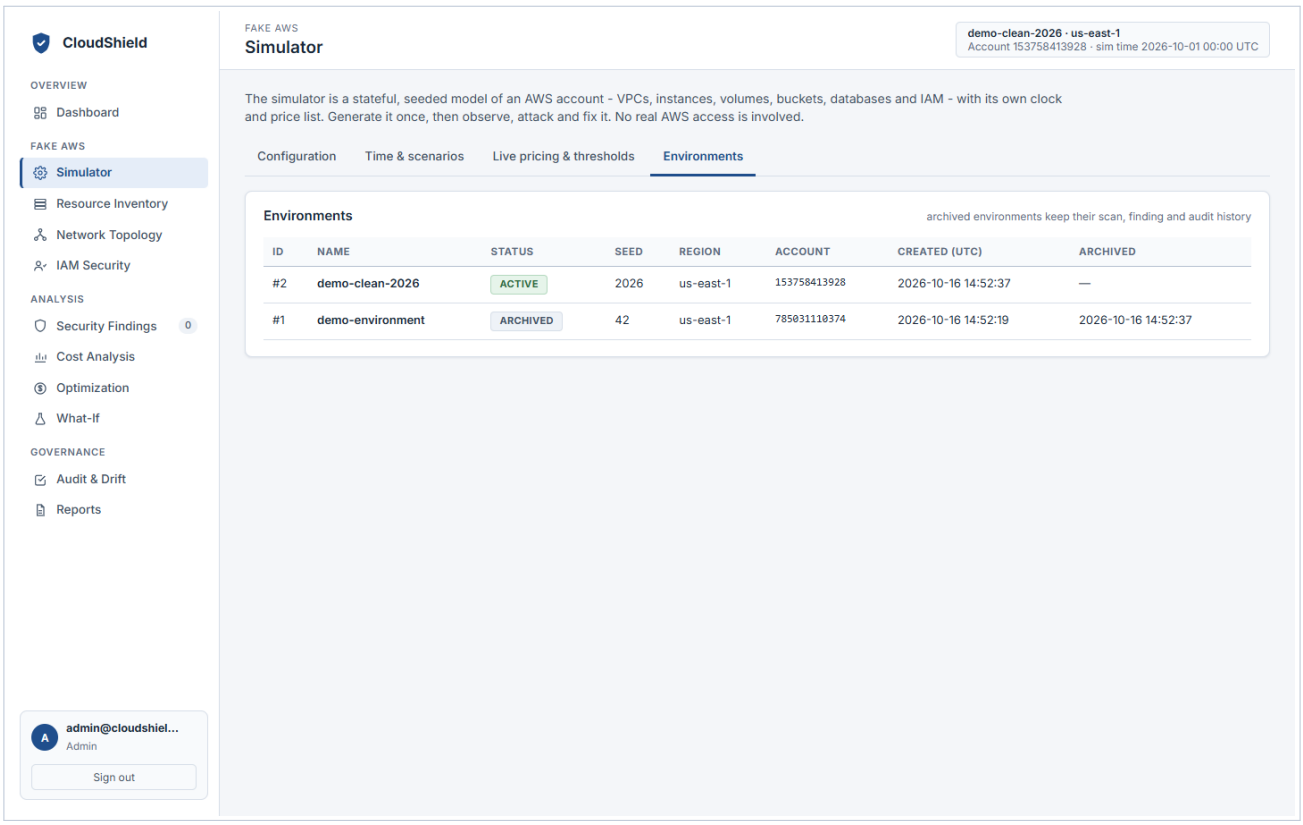


Figure 21. Environment history. The earlier environment is ARCHIVED, not erased; scans and audit history remain.

Inventory, topology and IAM

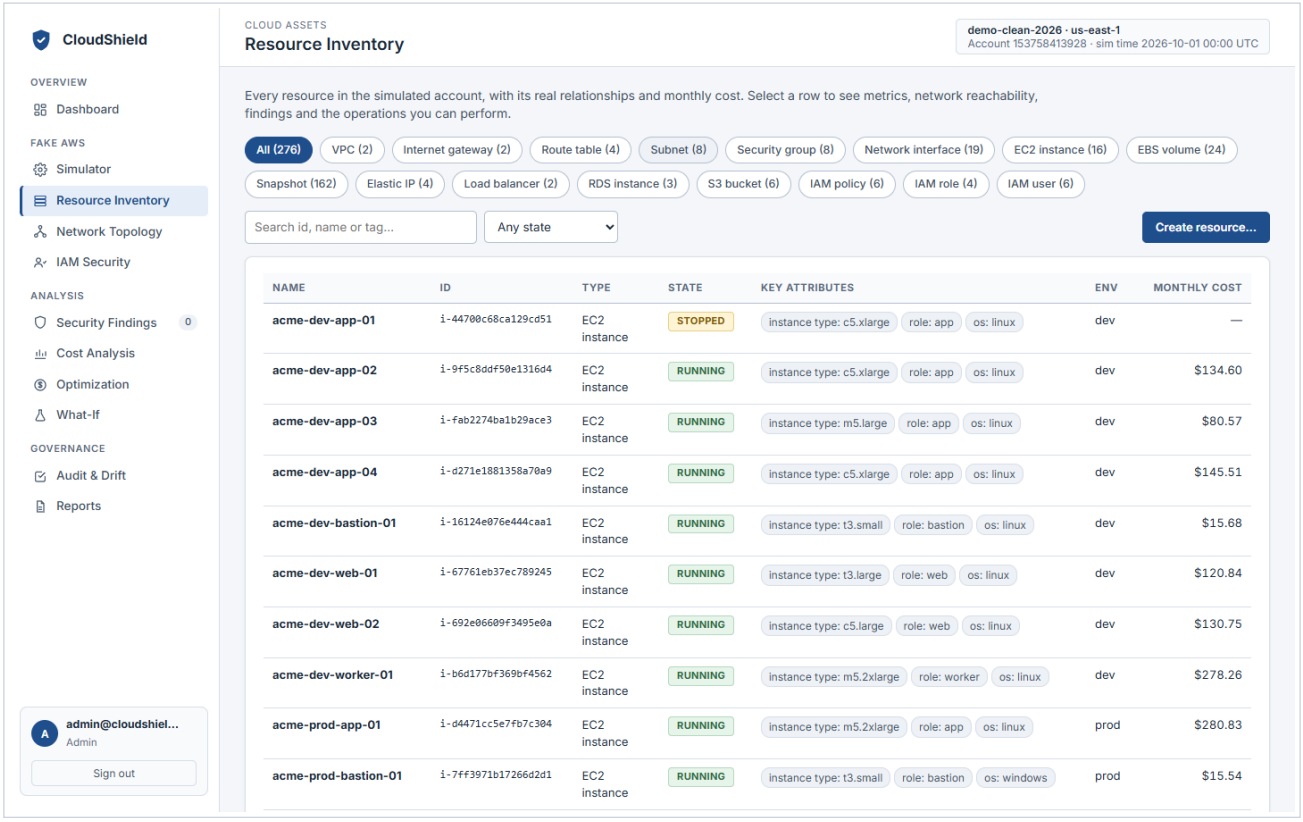


Figure 22. Resource inventory. Paginated inventory of every simulated resource with state, key attributes, environment and monthly cost.

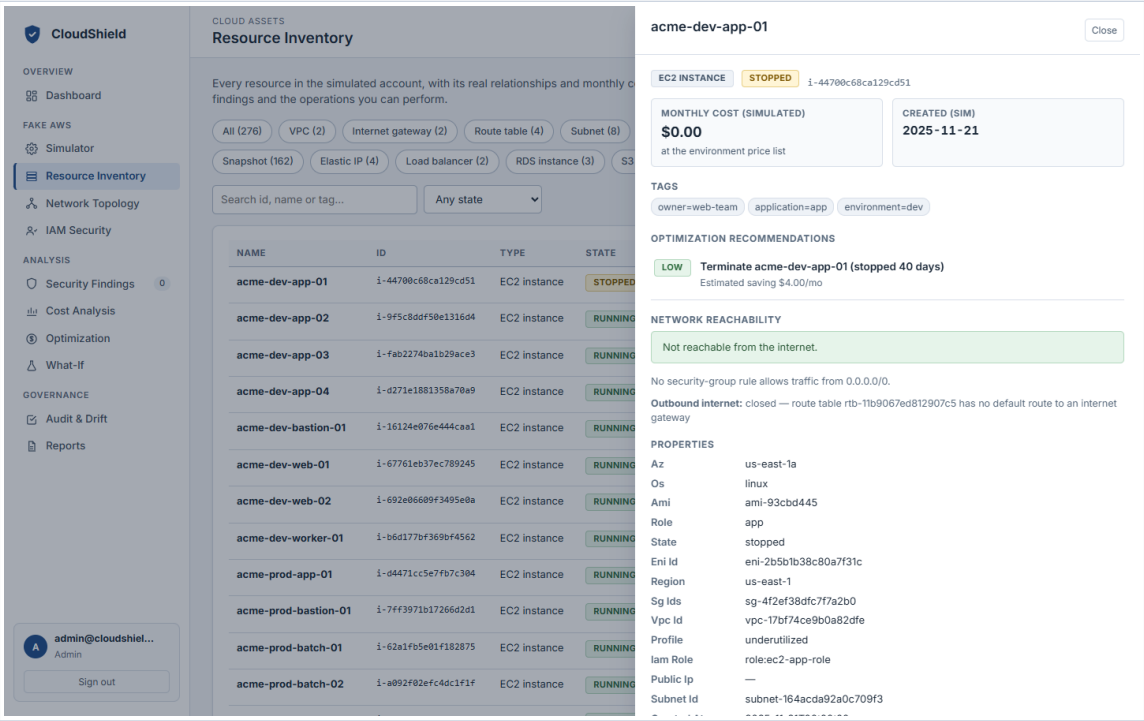


Figure 23. Resource detail drawer. Per-resource properties, tags, metrics, relationships, network reachability explanation and the validated operations available.

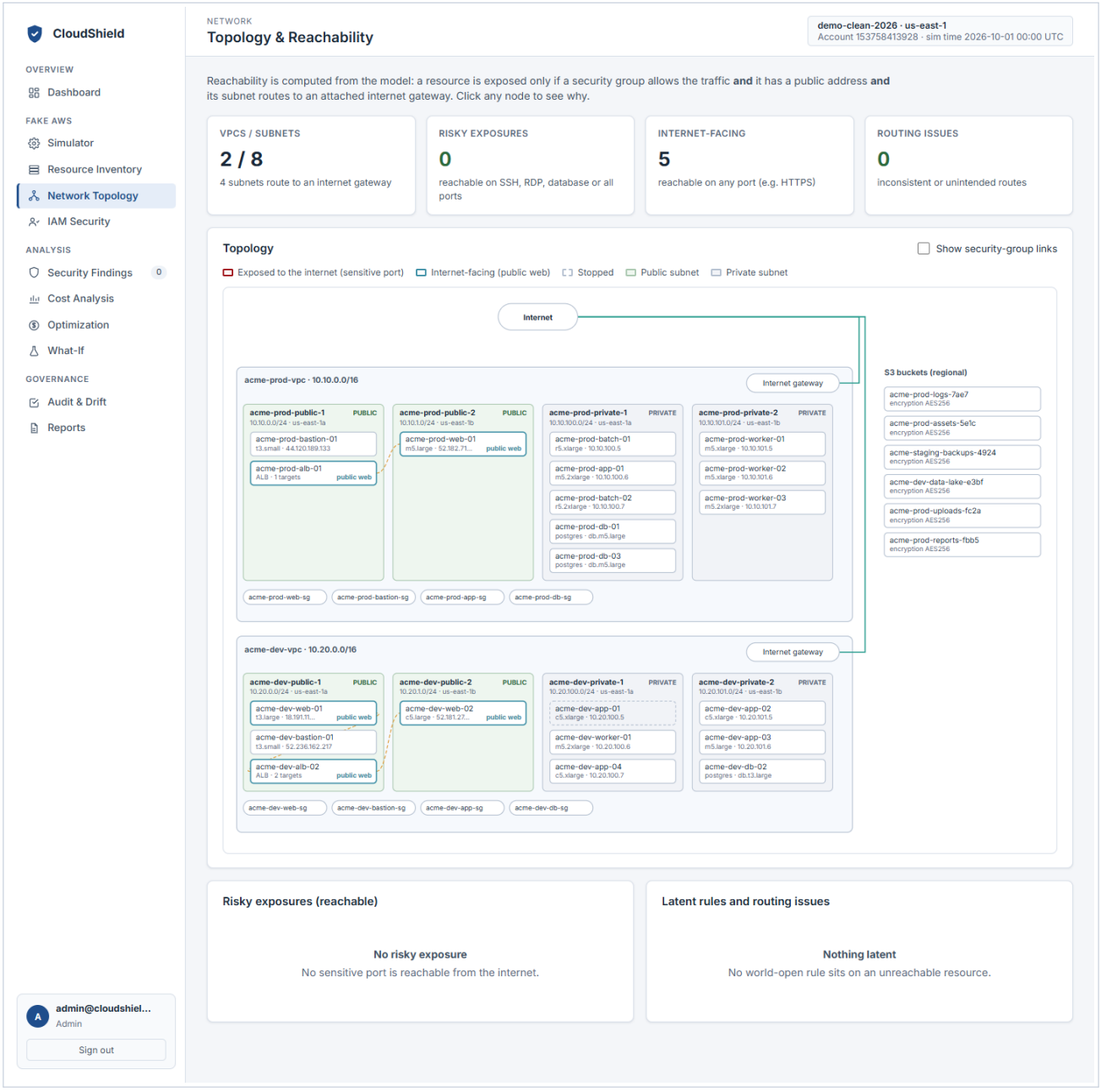


Figure 24. Network topology (no exposure). VPCs, public/private subnets, internet gateways, instances, databases and security groups. Nothing is exposed on a sensitive port.

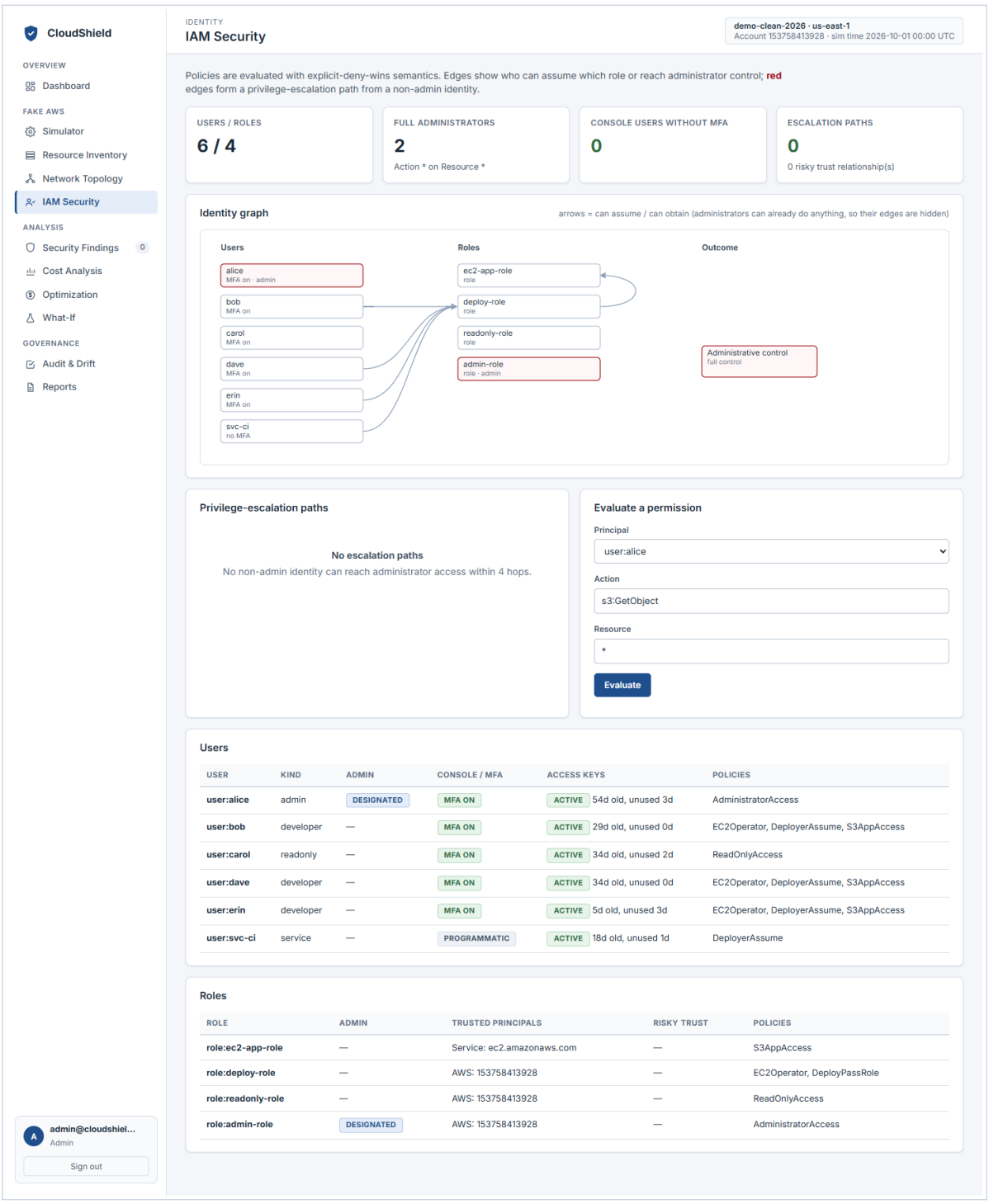


Figure 25. IAM identity analysis. Principals, admin detection, MFA and access-key status, trust relationships and the escalation graph.

Simulated time and scenarios

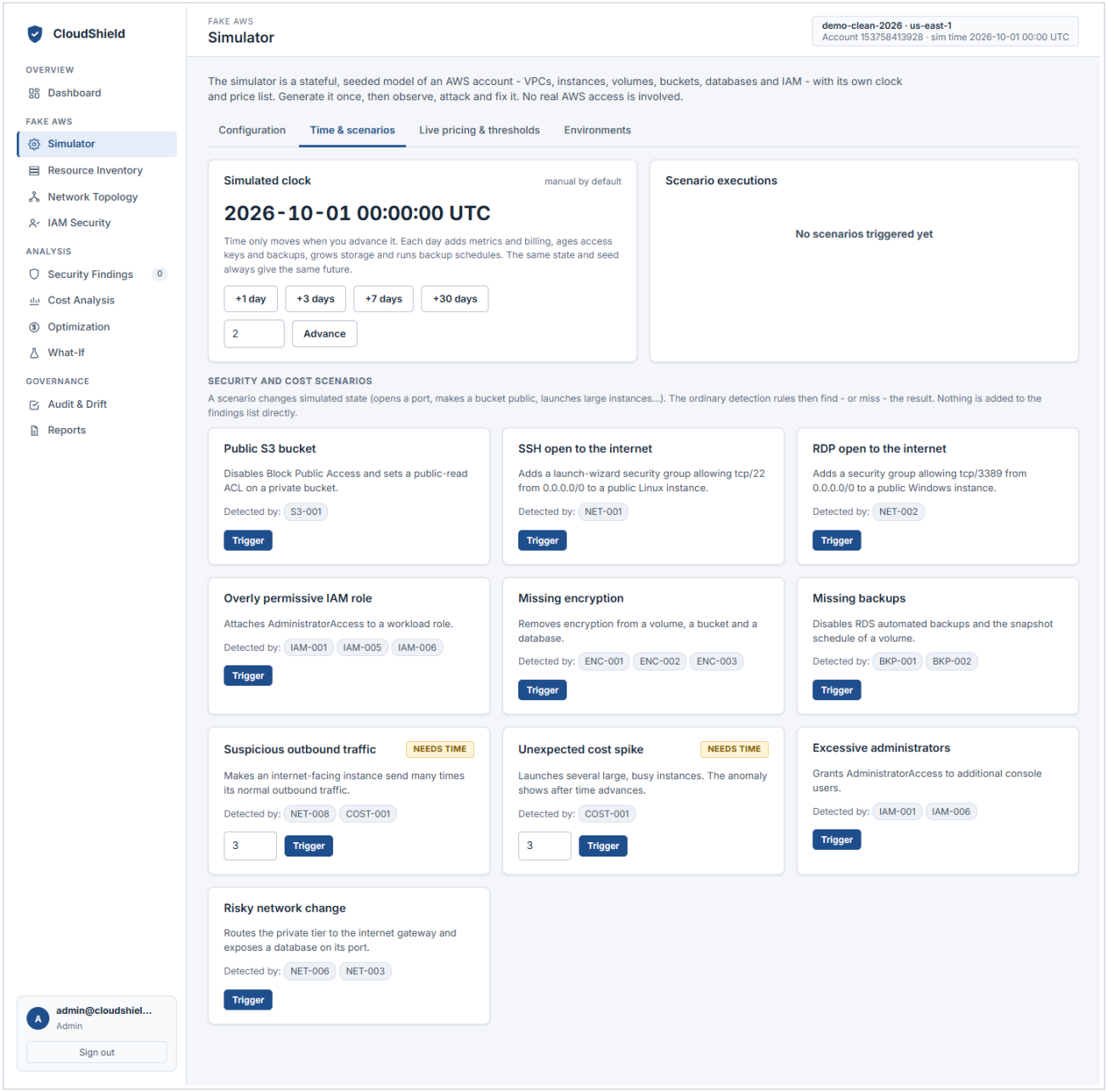


Figure 26. Simulated time and scenarios. Manual clock controls and the ten scenarios; each changes state and names the rules expected to detect it.

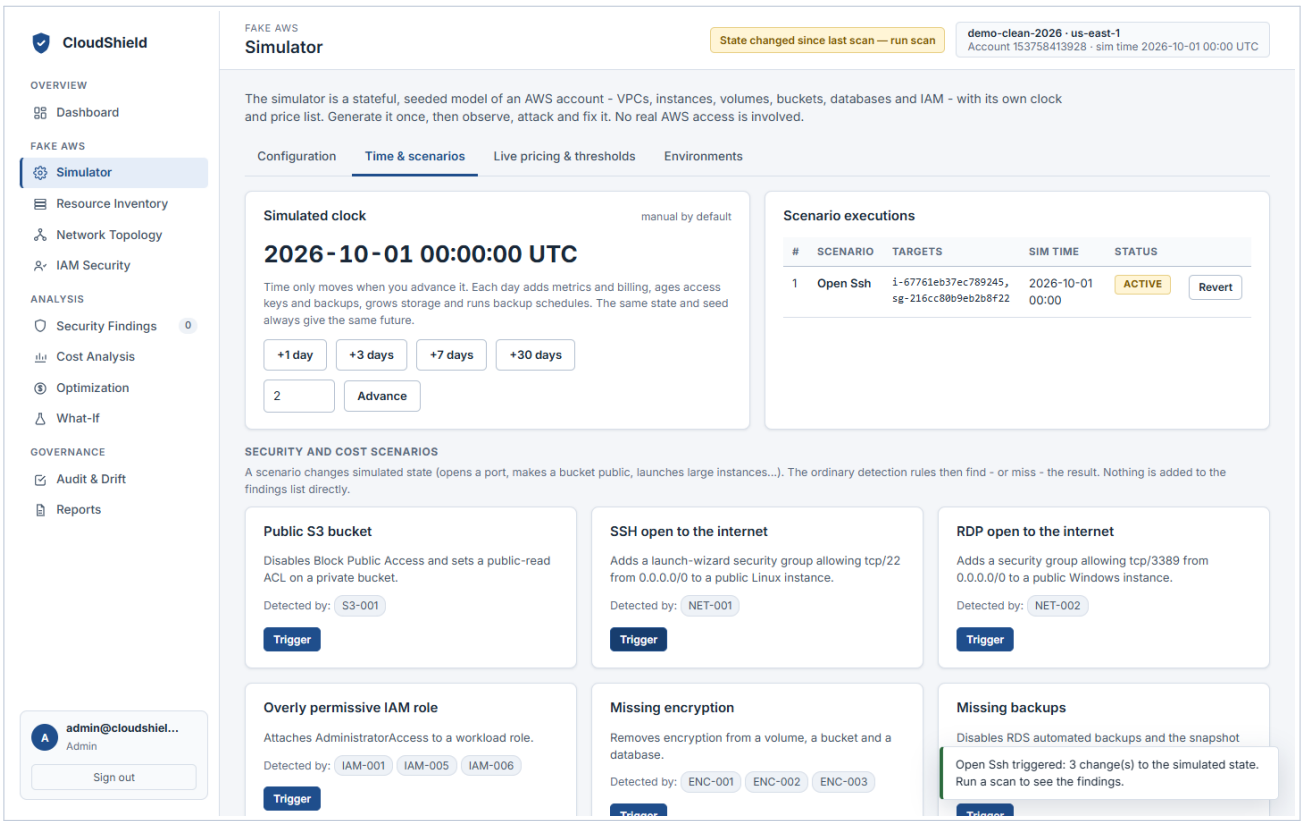


Figure 27. Scenario applied. State changed (a world-open SSH rule was added); the 'state changed since last scan' chip appears because no finding exists until a scan runs.

Detection with evidence (before)

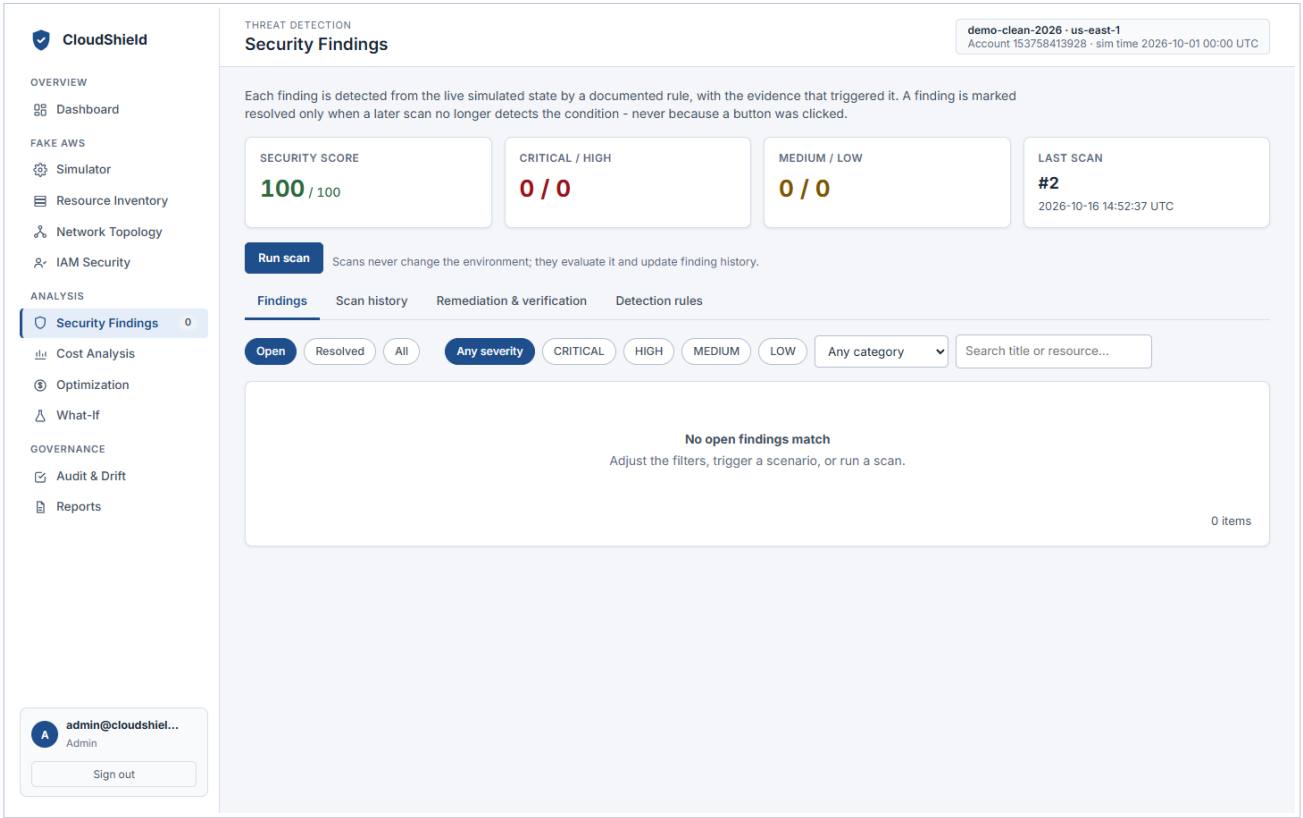


Figure 28. Findings on the clean environment. Before any scenario: no critical or high findings are detected.

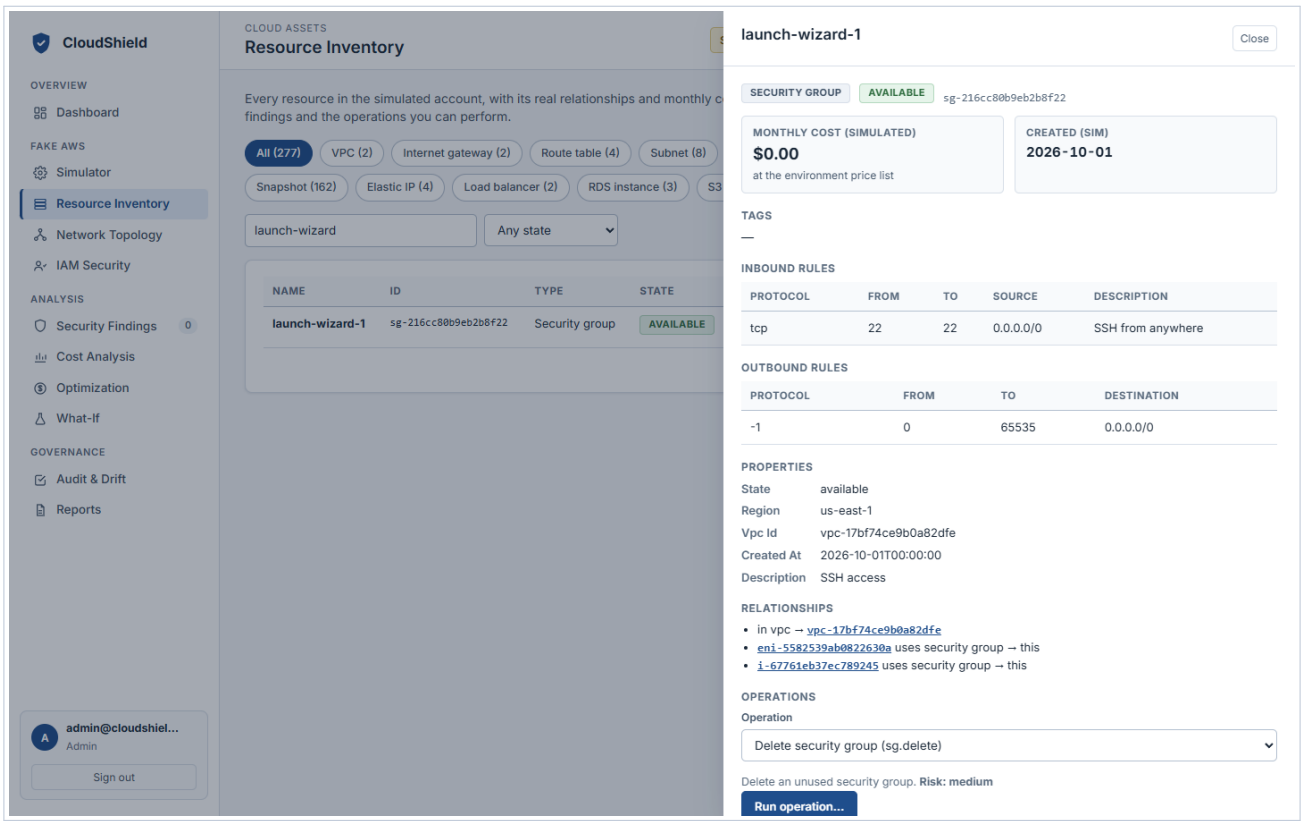


Figure 29. Security group BEFORE remediation. The scenario's security group allows tcp/22 from 0.0.0.0/0 (resource state, before the fix).

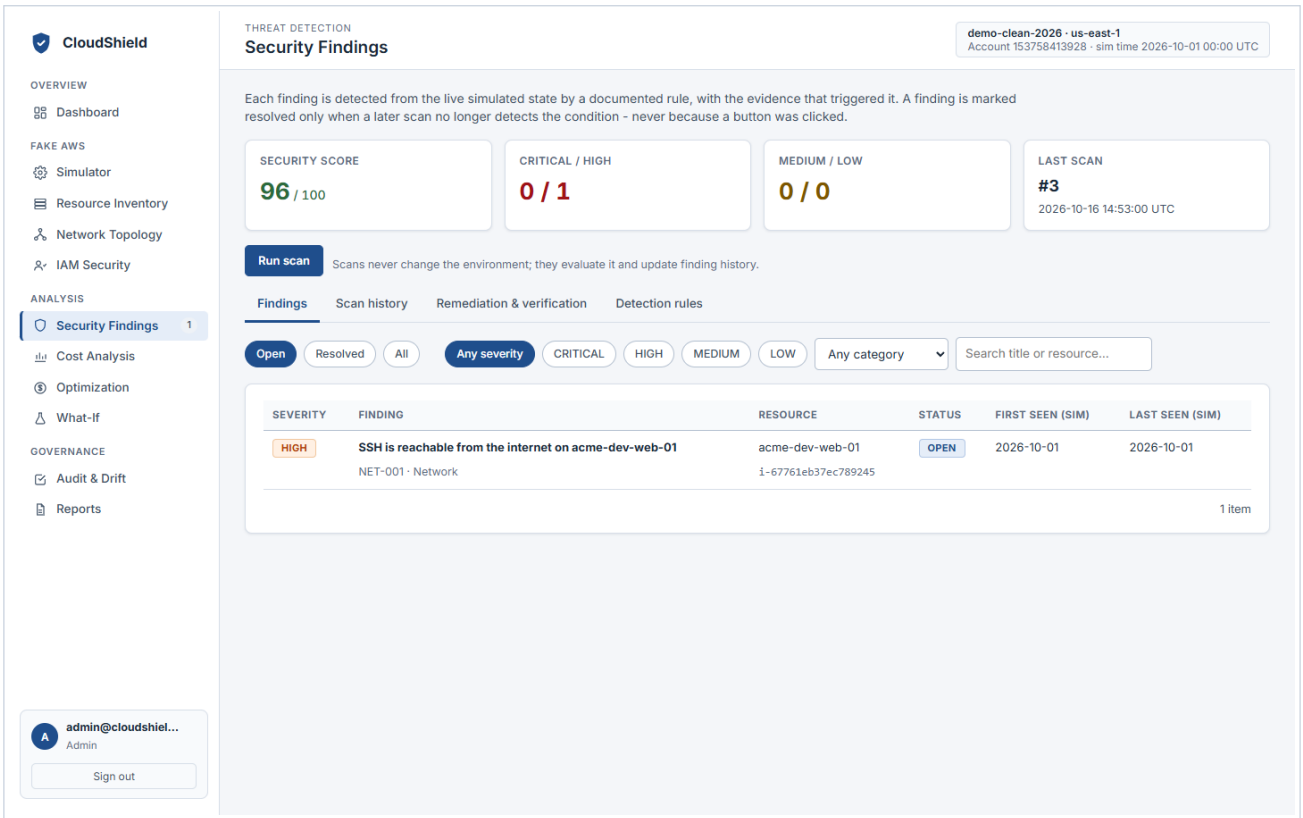


Figure 30. Findings after the scenario and a scan. A new HIGH finding NET-001 appears, derived from the changed security group, public address and route.

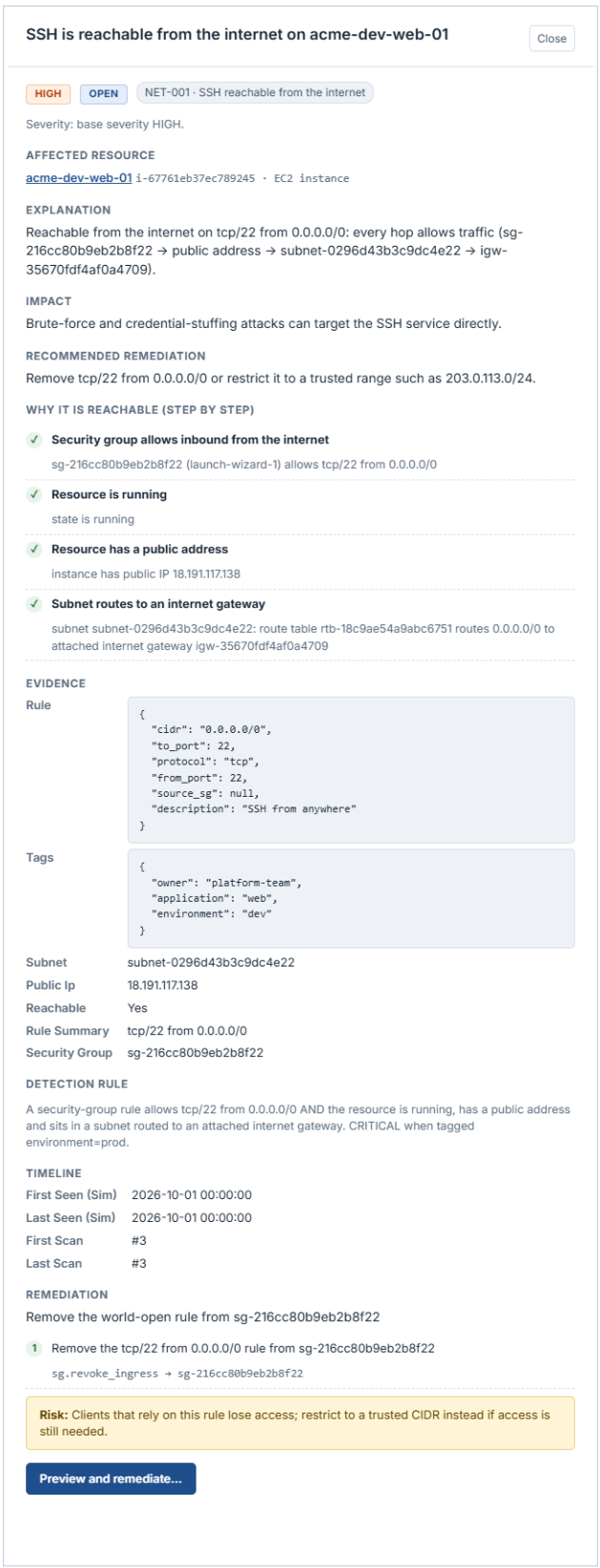


Figure 31. Finding with evidence. Explanation, impact, step-by-step reachability proof, detection rule, severity reason and timeline.

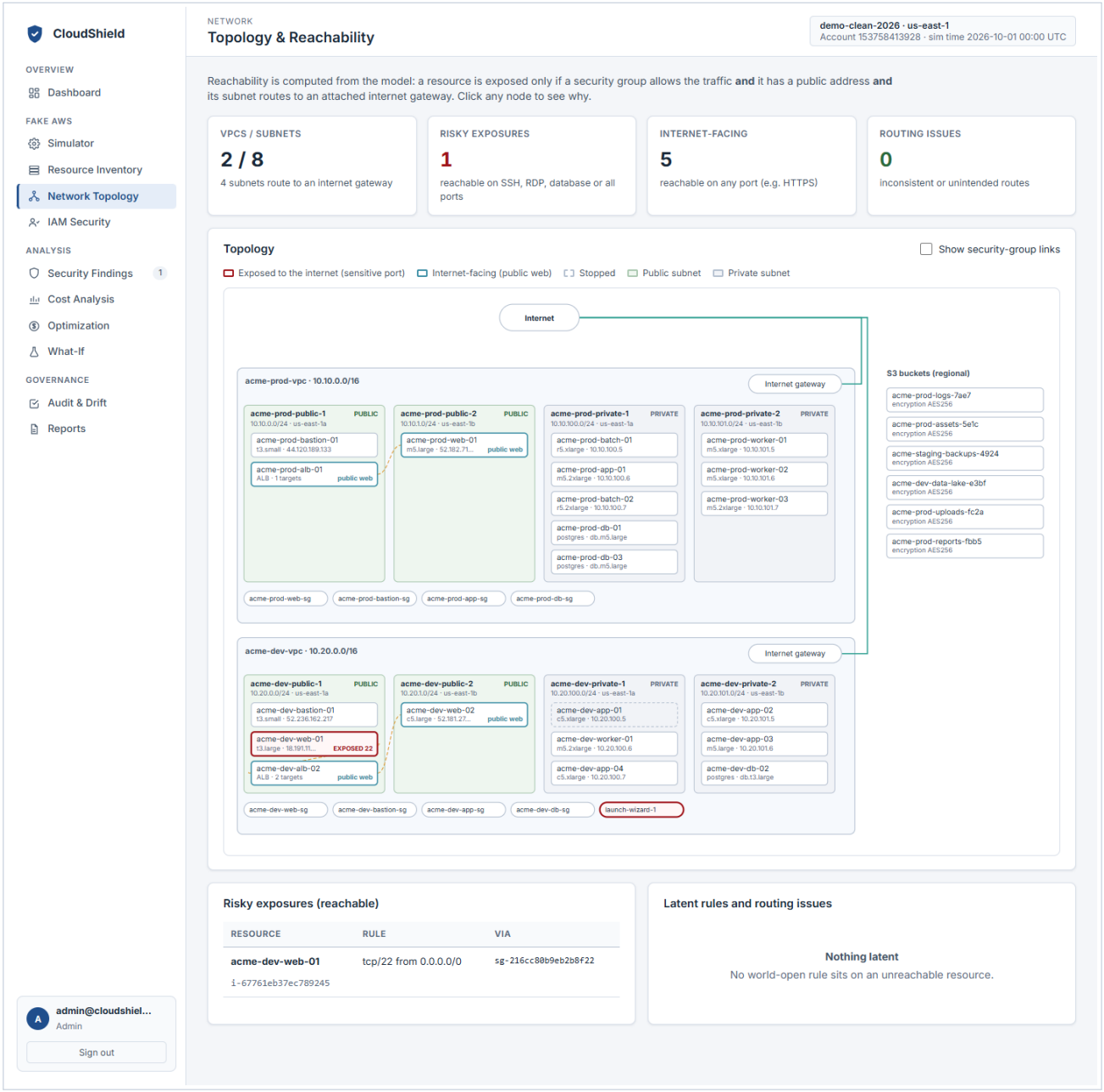


Figure 32. Network topology with an exposed instance. The instance is outlined red with 'EXPOSED 22' because the network model finds SSH reachable from the internet.

Remediation: preview, apply, verify, and the resource after the fix

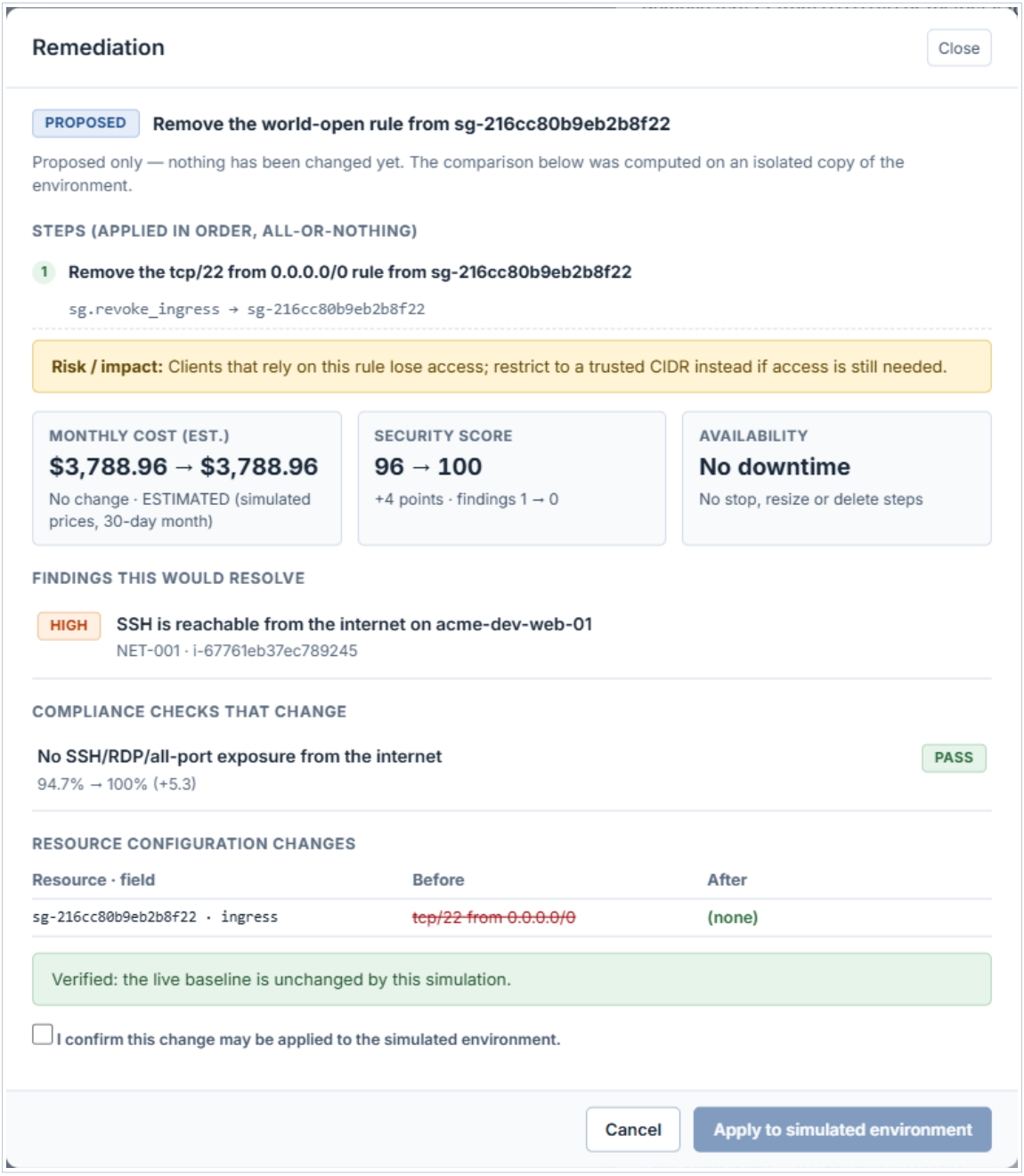


Figure 33. Remediation preview (PROPOSED). Plan, risk, and a what-if comparison computed on an isolated copy: score before/after, cost, availability, config diff. Nothing has changed yet.

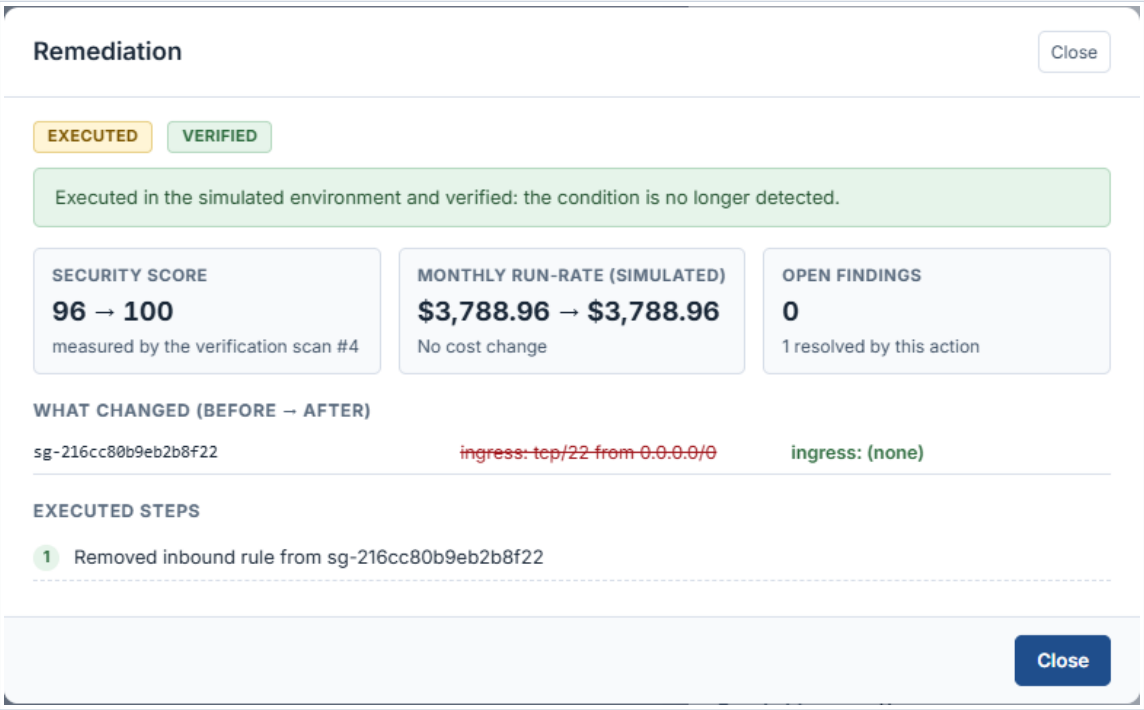


Figure 34. Remediation executed and verified. The change was applied to the simulated state, the analysis re-ran, and the condition is no longer detected (VERIFIED).

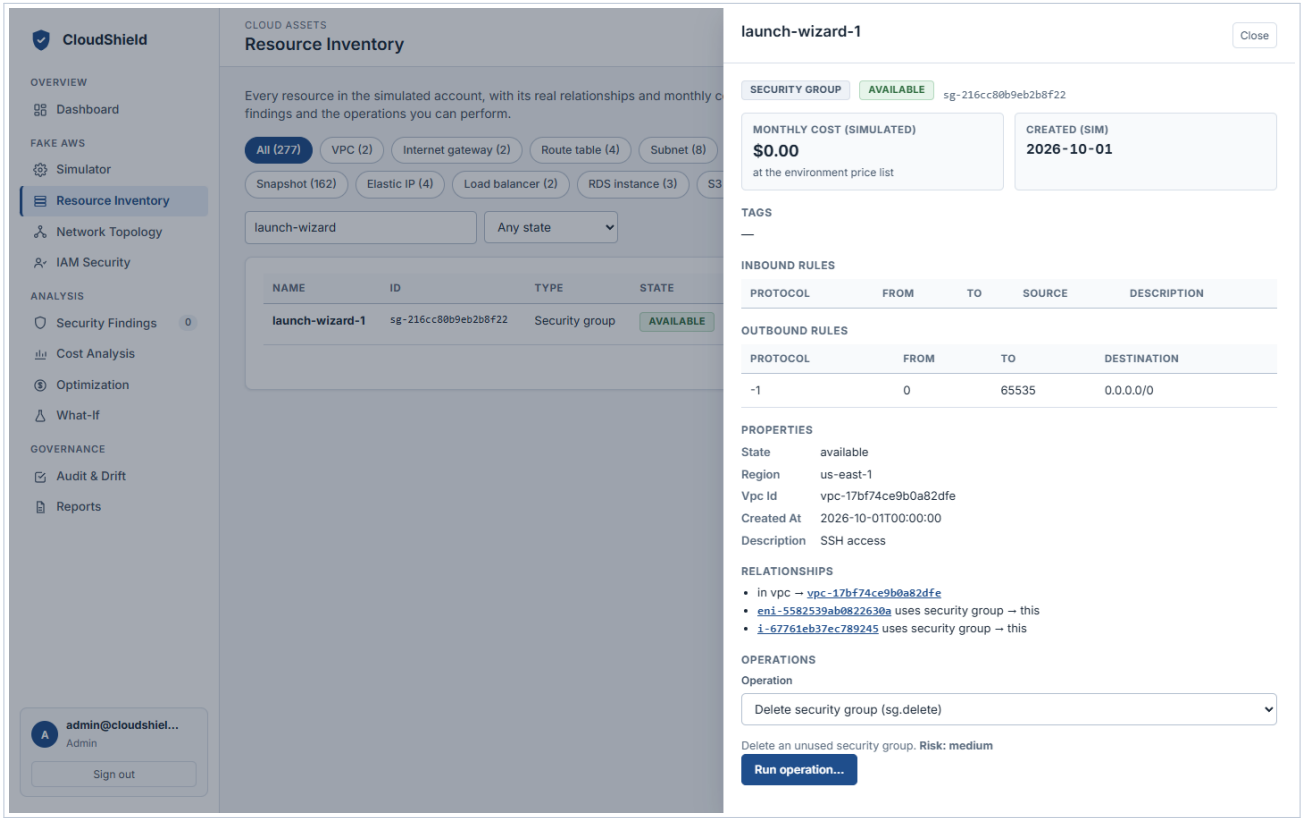


Figure 35. Security group AFTER remediation. Same resource after remediation: the world-open SSH rule is gone (compare with the BEFORE screenshot).

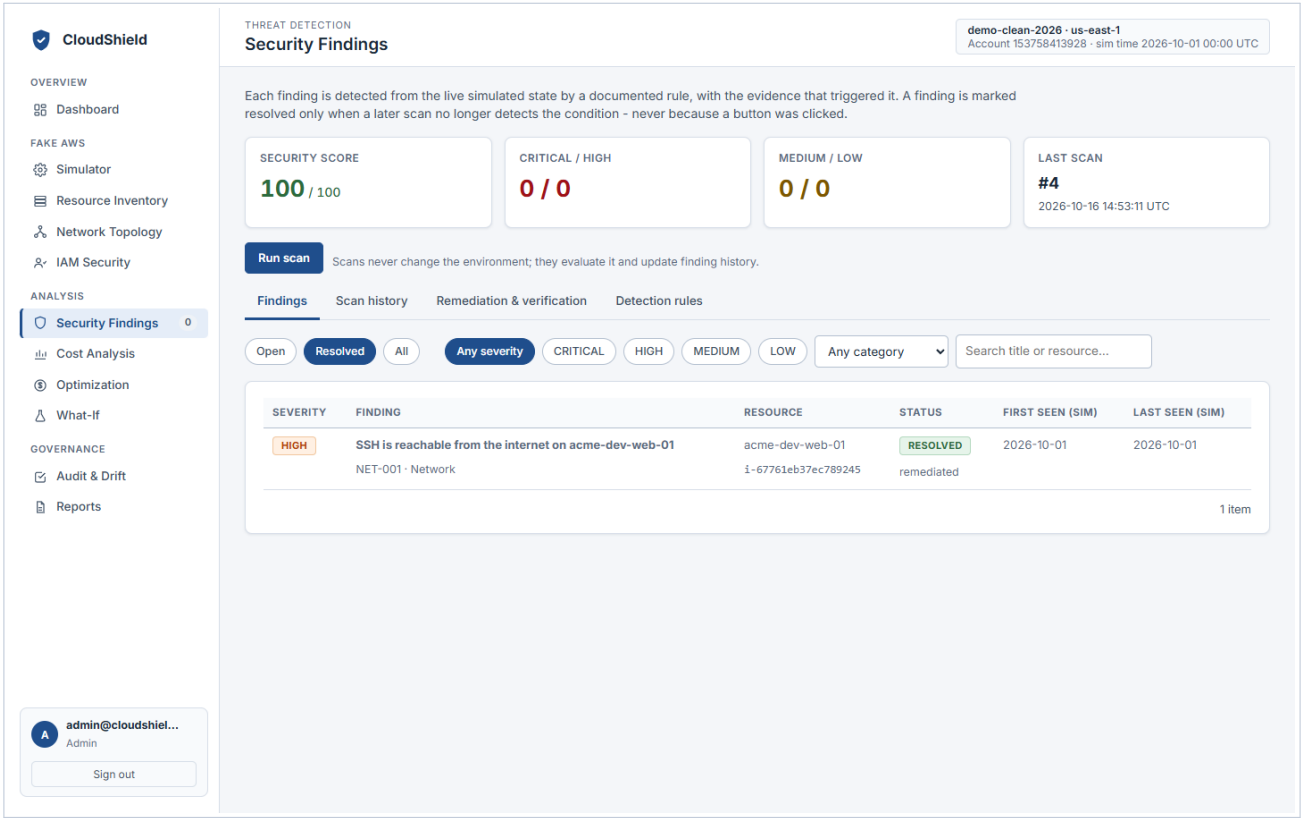


Figure 36. Resolved finding kept in history. The finding is RESOLVED because a re-scan no longer detects it; the record and its history remain.

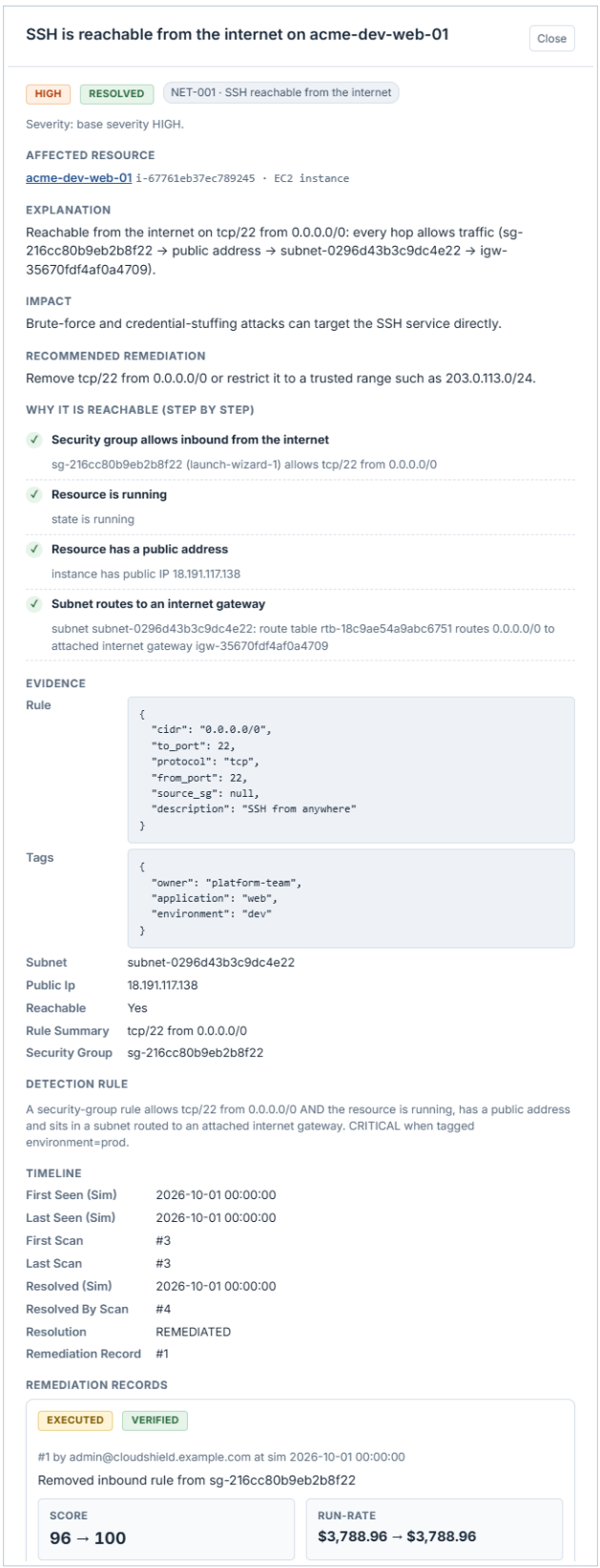


Figure 37. Resolved finding detail. Timeline, resolving scan, remediation record and verification result.

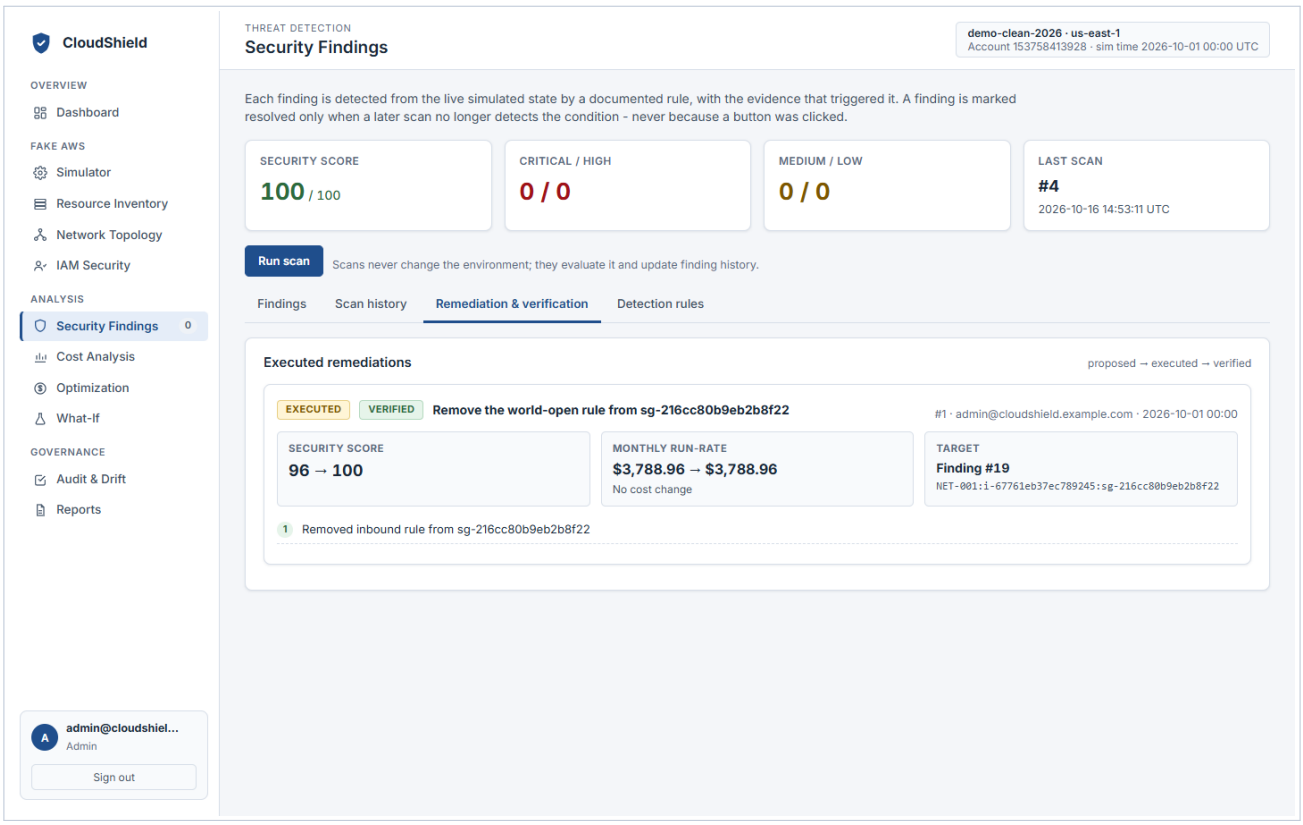


Figure 38. Remediation and verification history. Proposed -> executed -> verified, with score and run-rate before/after.

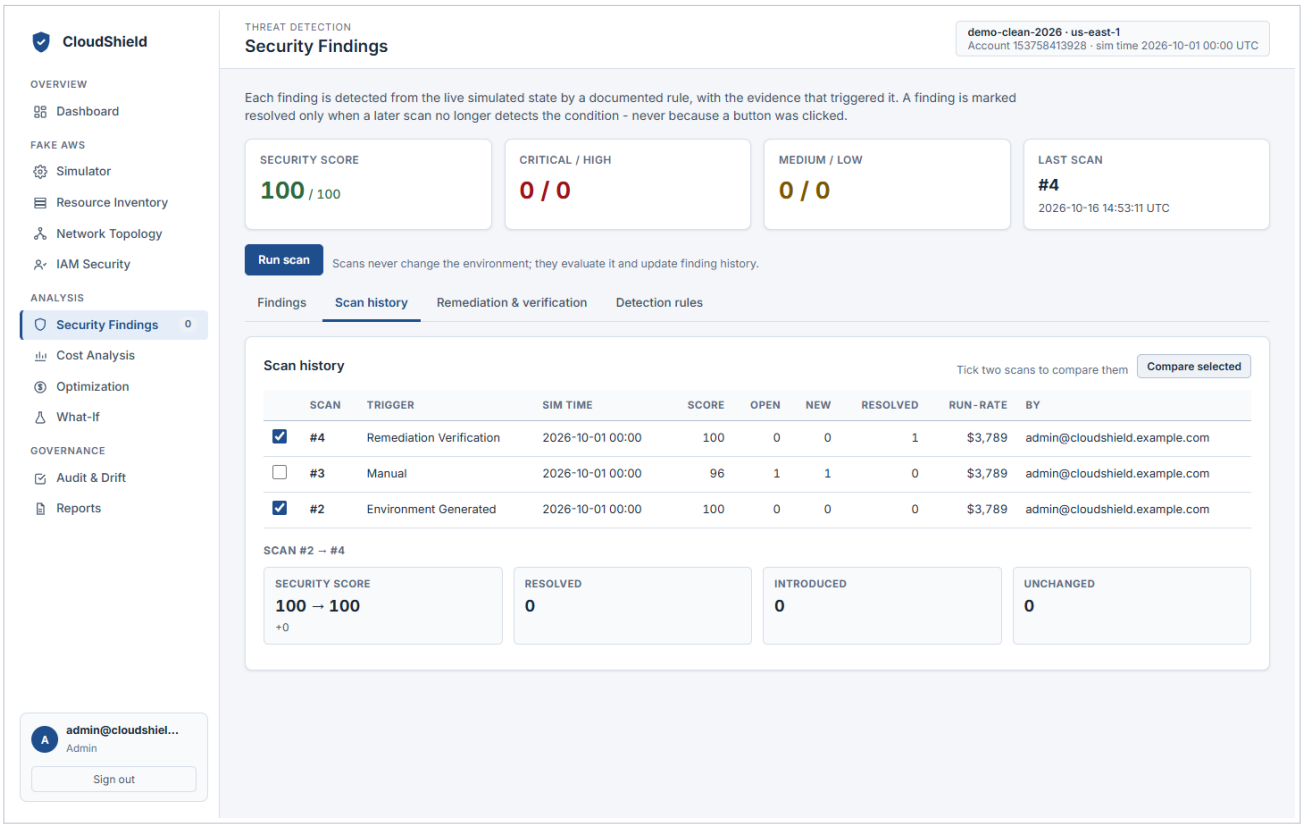


Figure 39. Scan history and comparison. Compare two scans: resolved/introduced findings and score change.

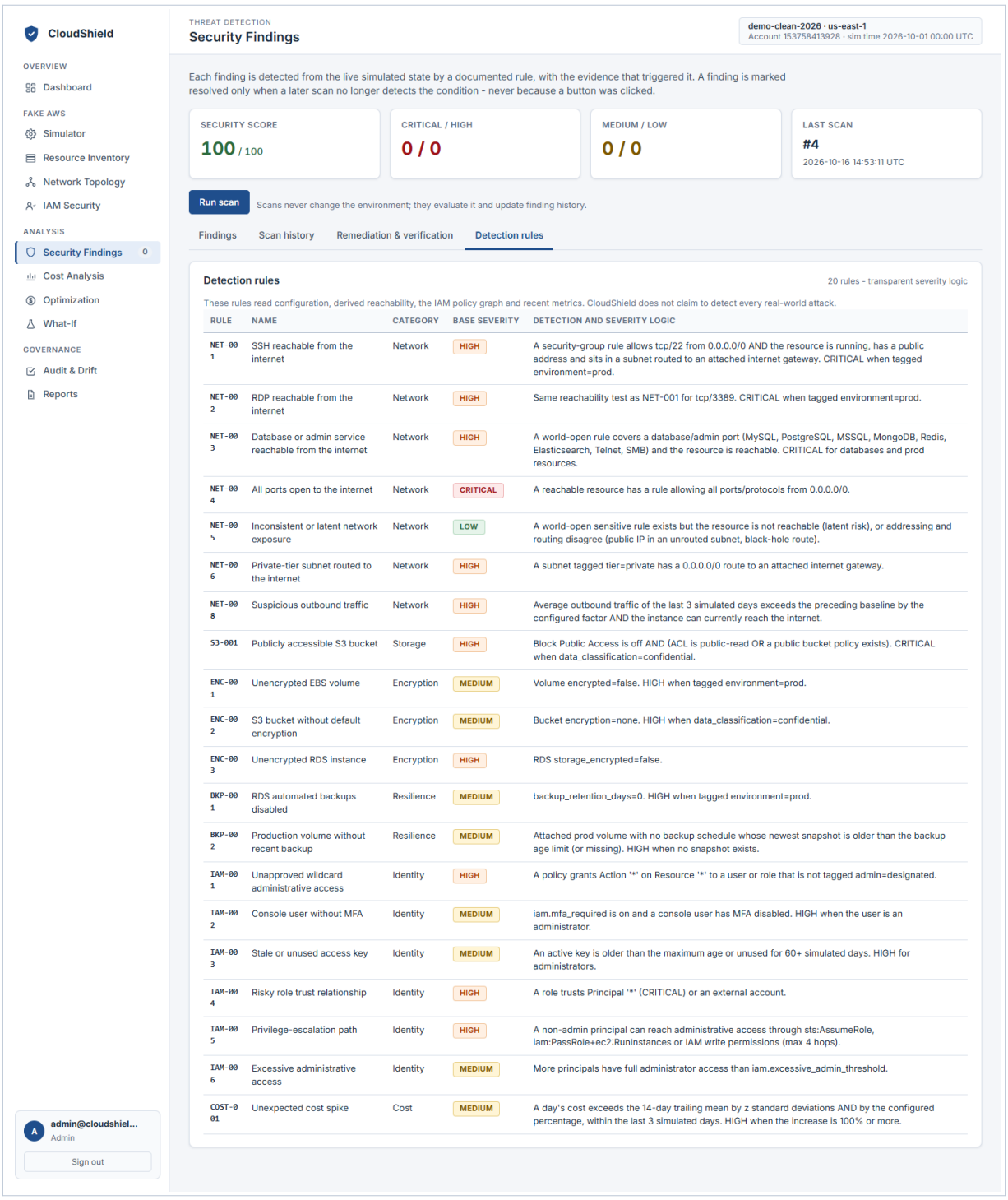


Figure 40. Detection rules catalogue. Each rule's logic and severity basis is documented and visible.

IAM privilege escalation

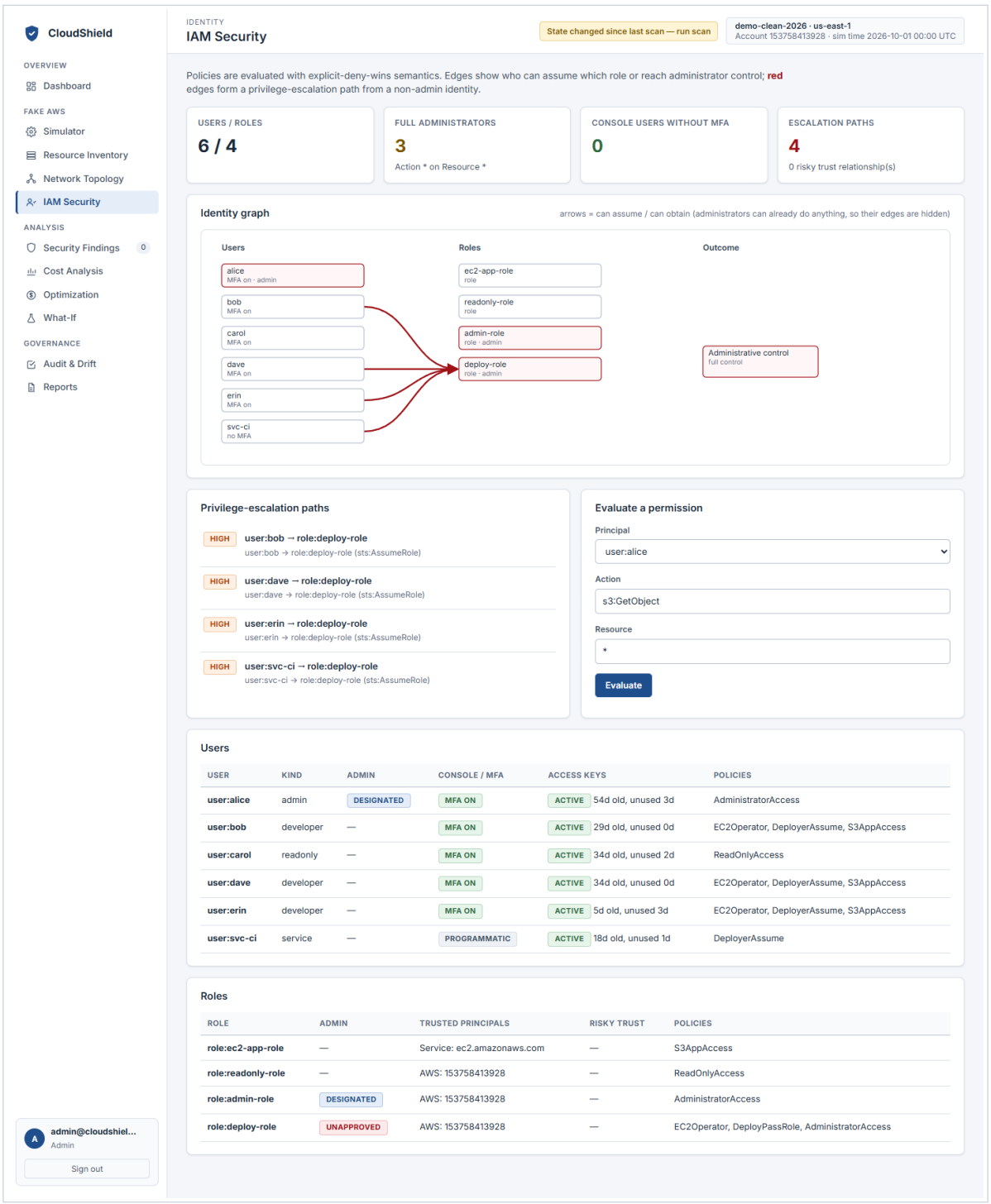


Figure 41. IAM privilege escalation after a permissive role. AdministratorAccess on a workload role creates escalation paths from ordinary users; red edges show the chain.

Cost analysis and optimization

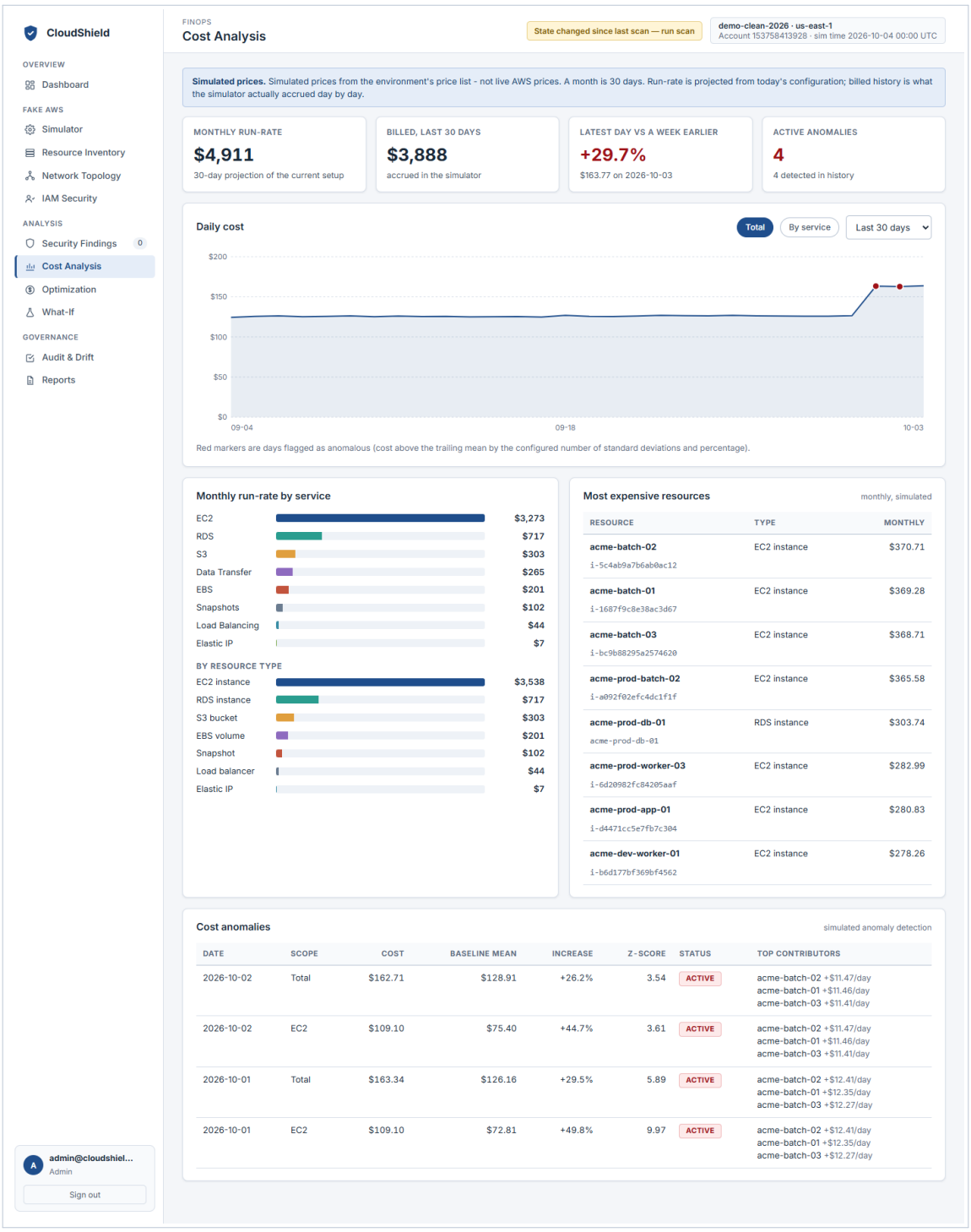


Figure 42. Cost analysis with a detected anomaly. Daily cost chart with the spike flagged (z-score above baseline), by-service breakdown and contributors.

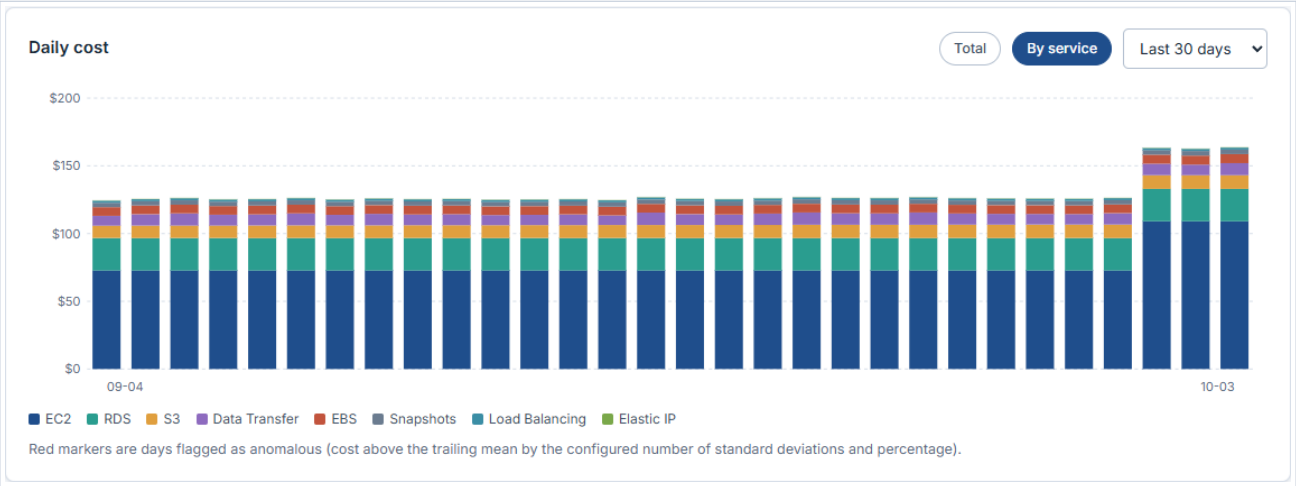


Figure 43. Daily cost by service. Stacked daily cost per service from simulated billing.

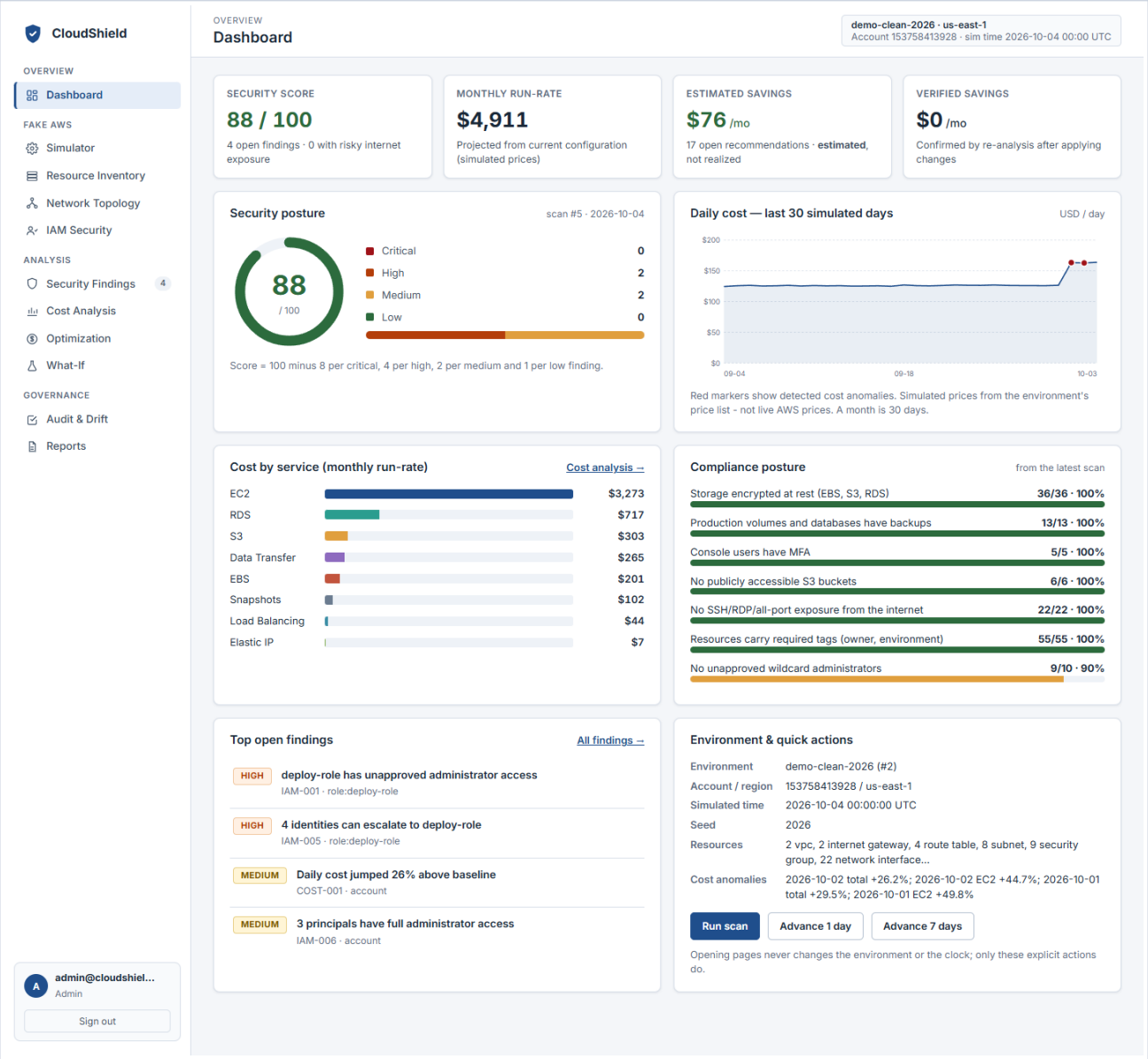


Figure 44. Dashboard after scenarios and a rescan. Dashboard reflects the new findings, the cost anomaly and updated compliance posture.

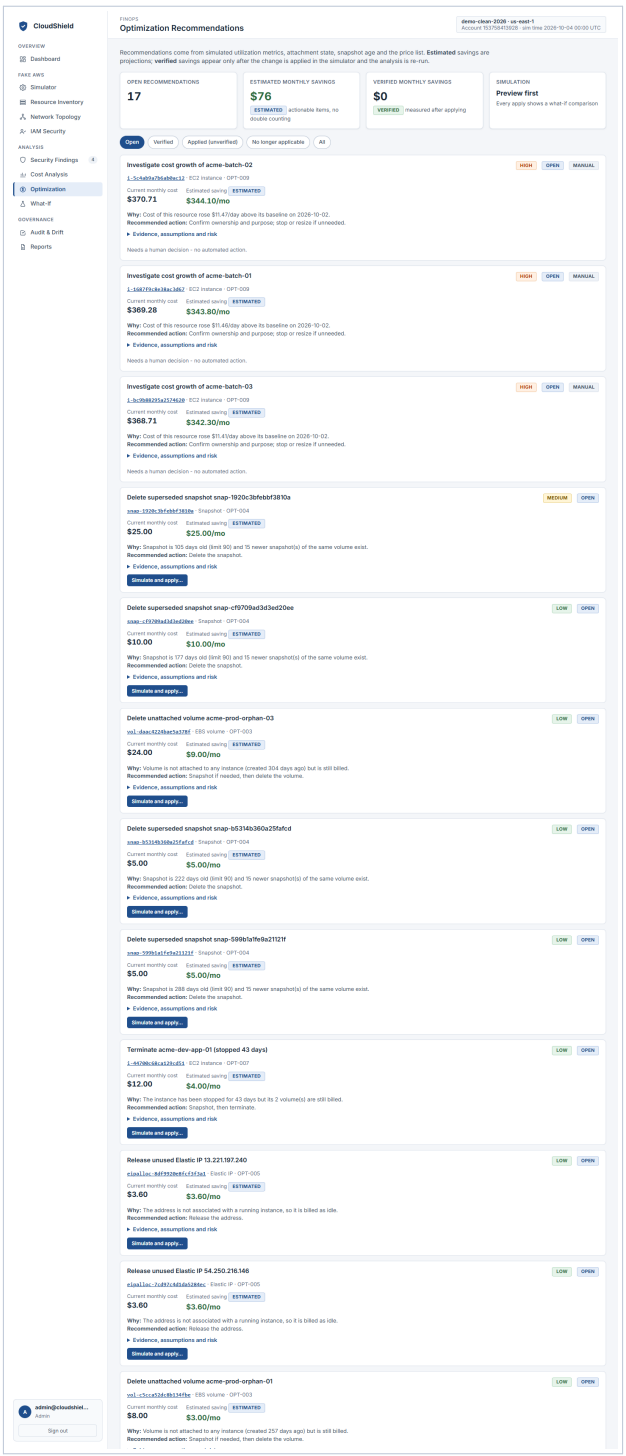


Figure 45. Optimization recommendations. Each recommendation shows evidence, estimated saving (labelled ESTIMATED), risk and the action to take.

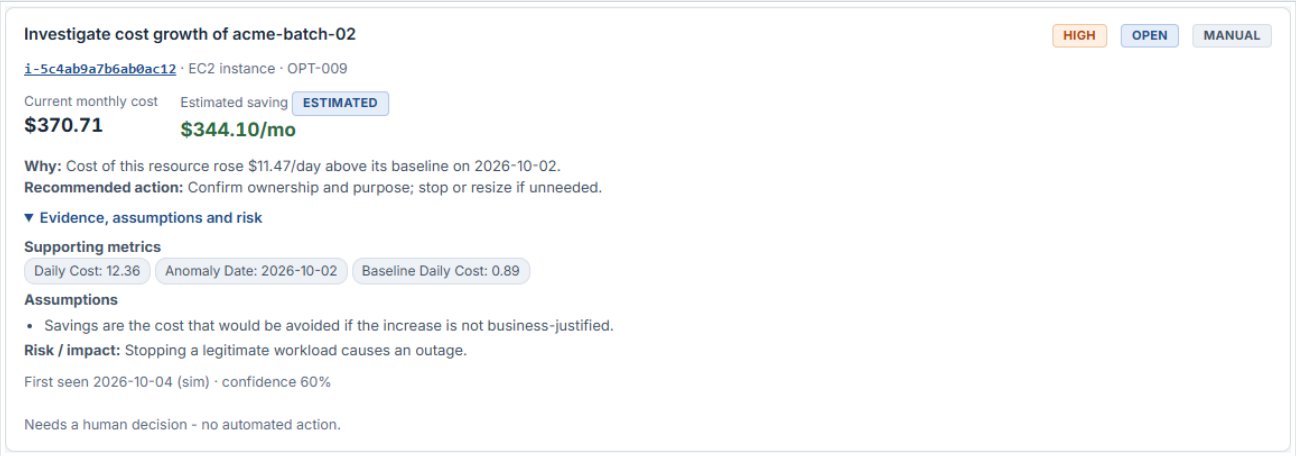


Figure 46. Recommendation evidence and assumptions. Supporting metrics, assumptions, risk and the executable plan.

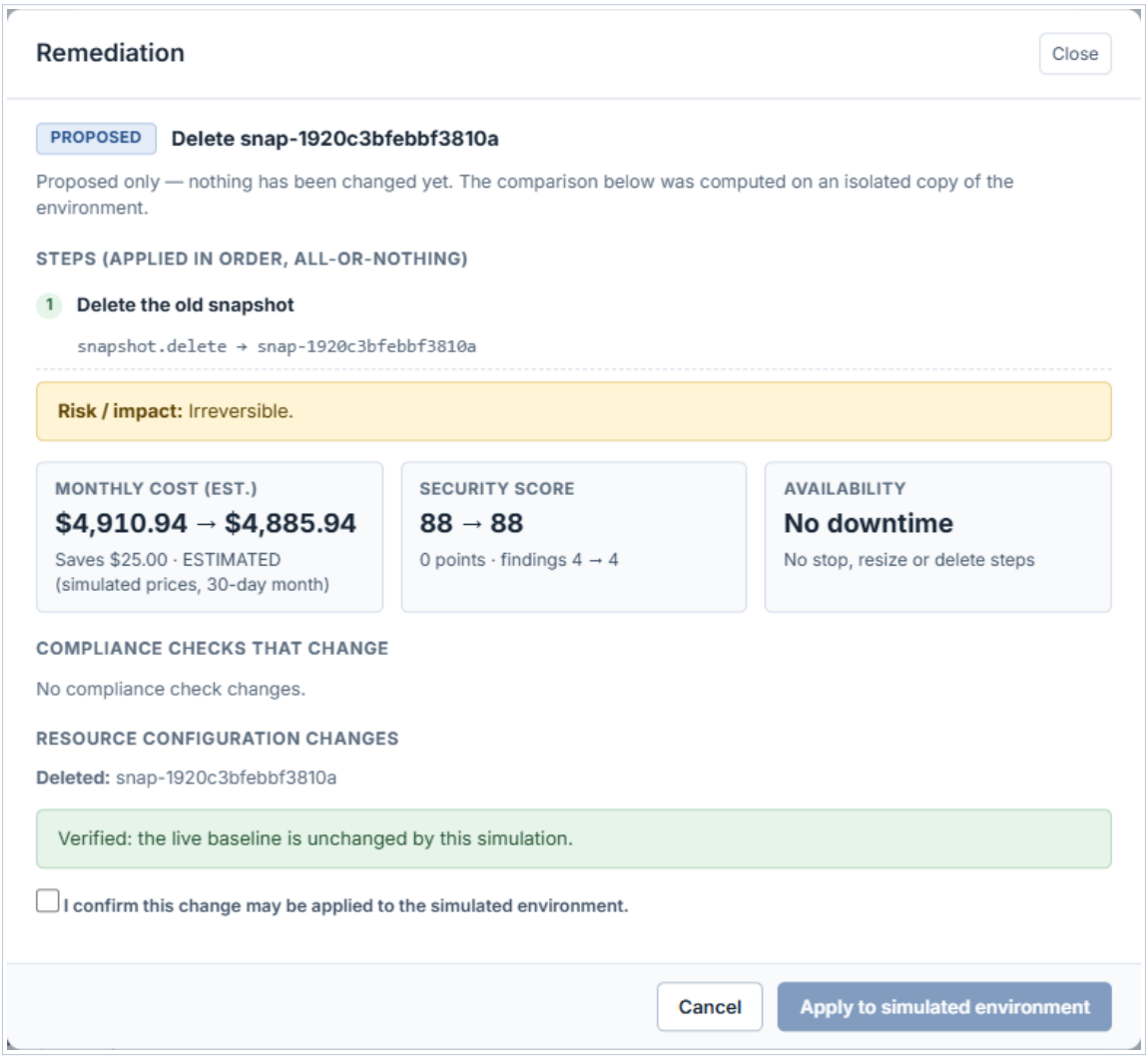


Figure 47. Recommendation what-if preview. Cost and security effect simulated before applying.

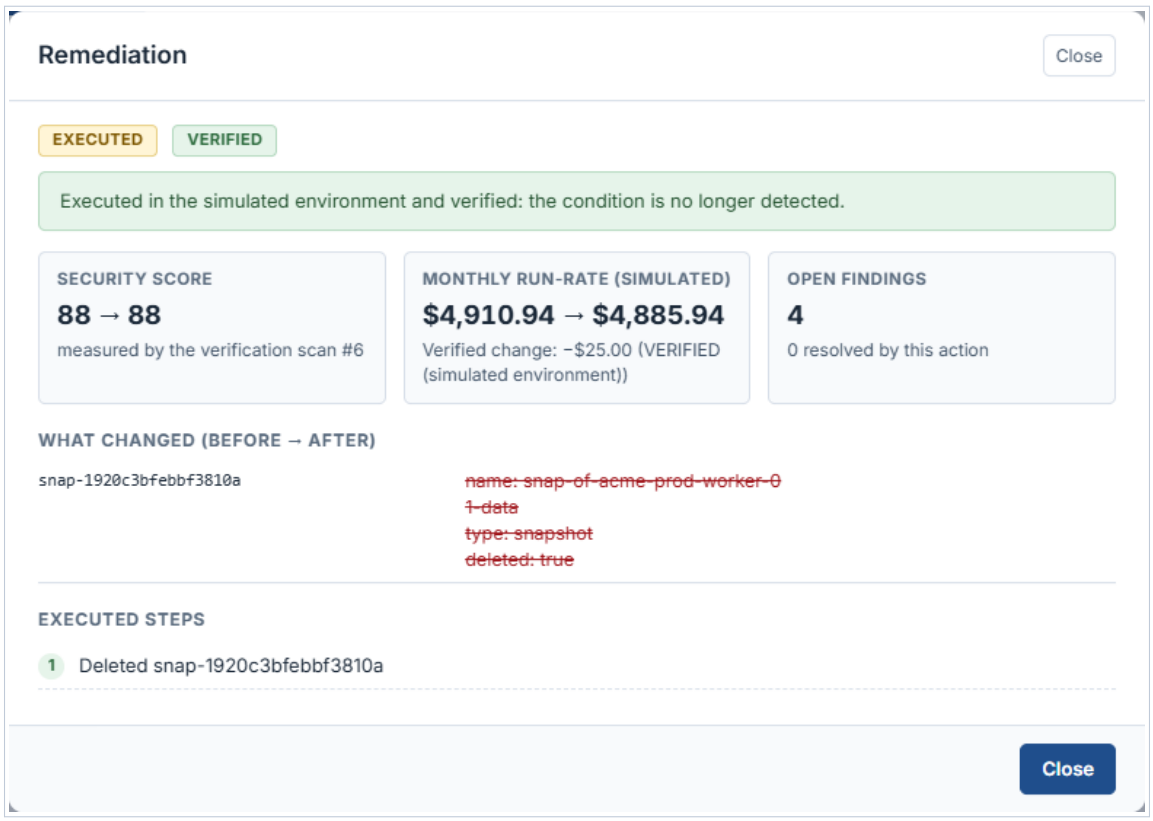


Figure 48. Recommendation applied and verified. Run-rate before/after shows the verified (simulated) saving.

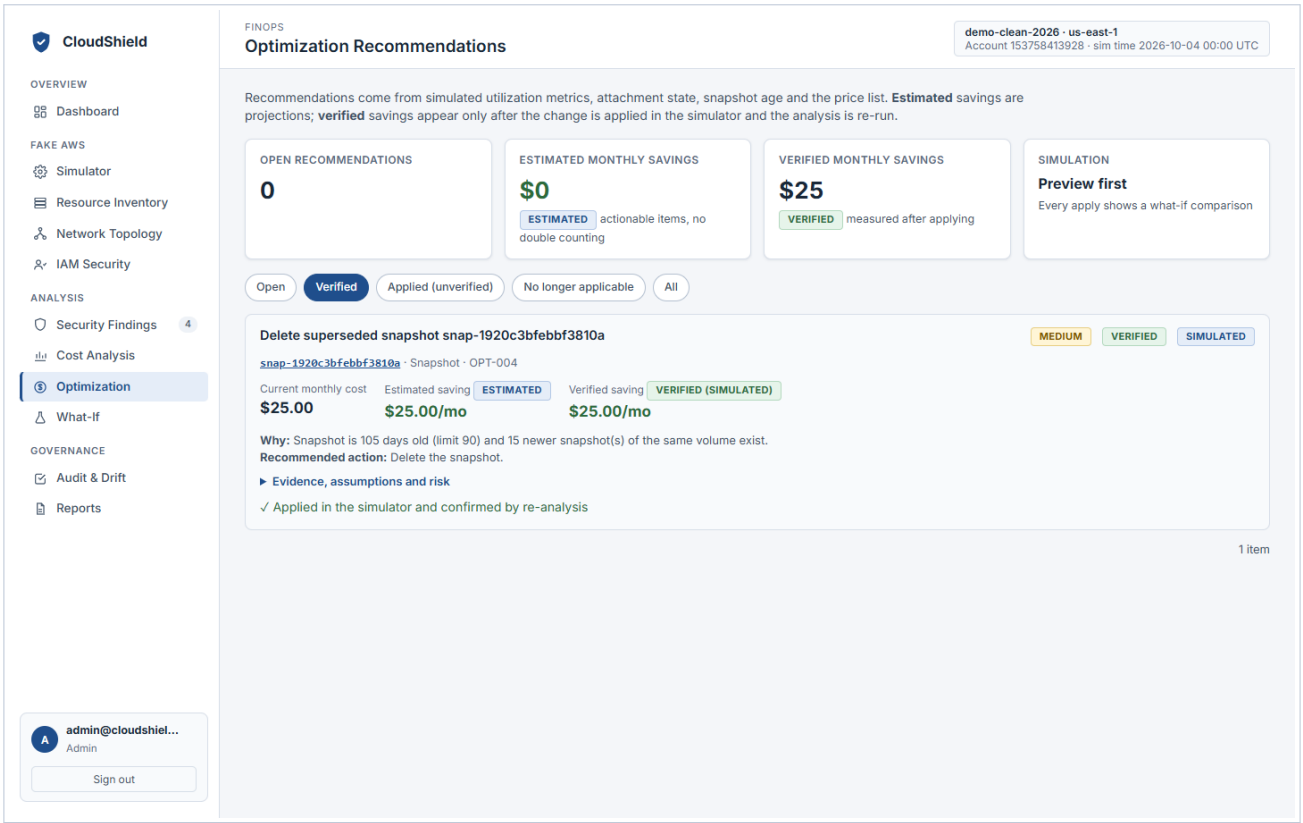


Figure 49. Verified recommendation. Estimated and verified savings are shown separately.

What-if, audit and drift

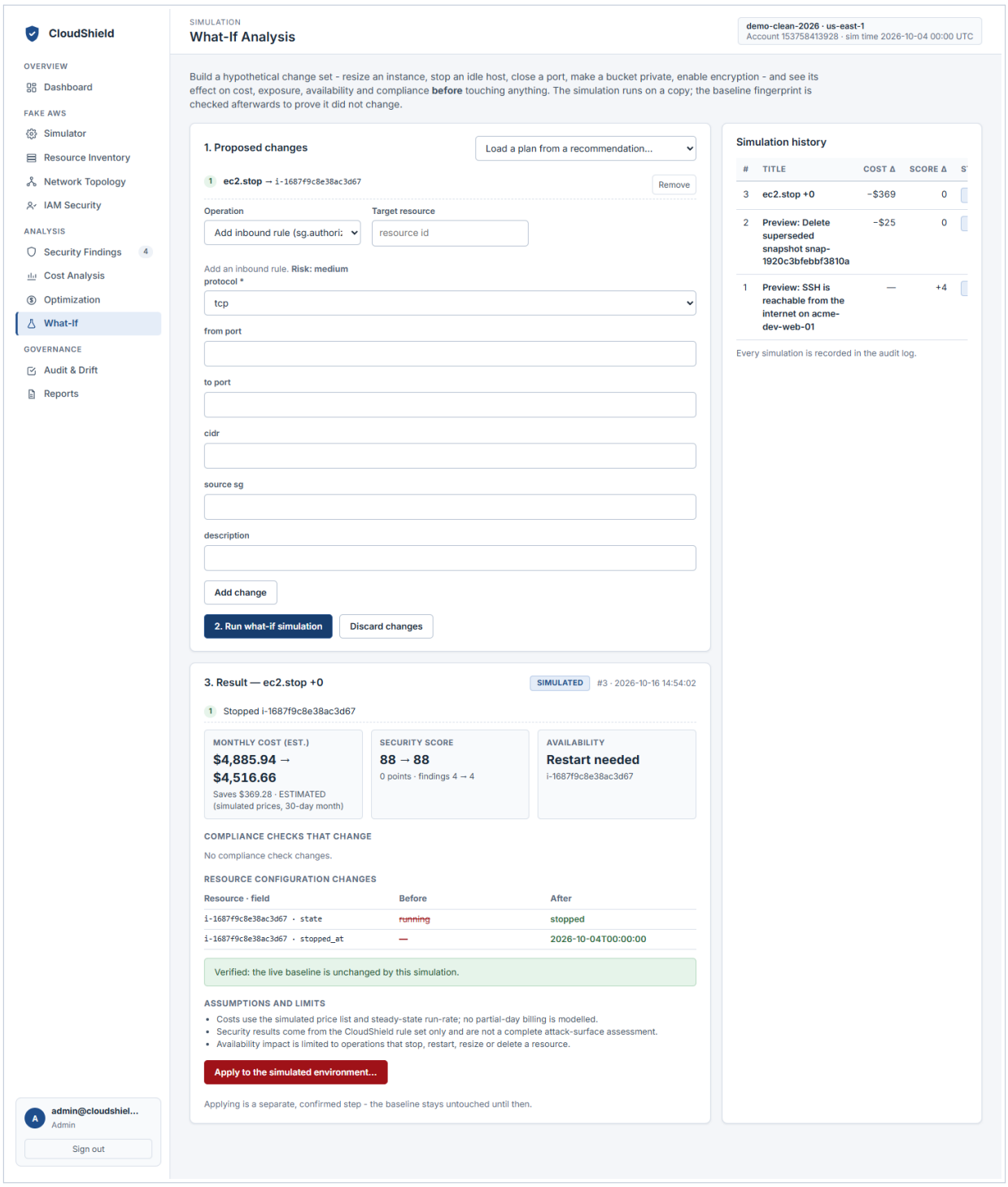


Figure 50. What-if comparison. Baseline vs hypothetical: cost, security score, availability, compliance and config diff; the baseline fingerprint is verified unchanged.

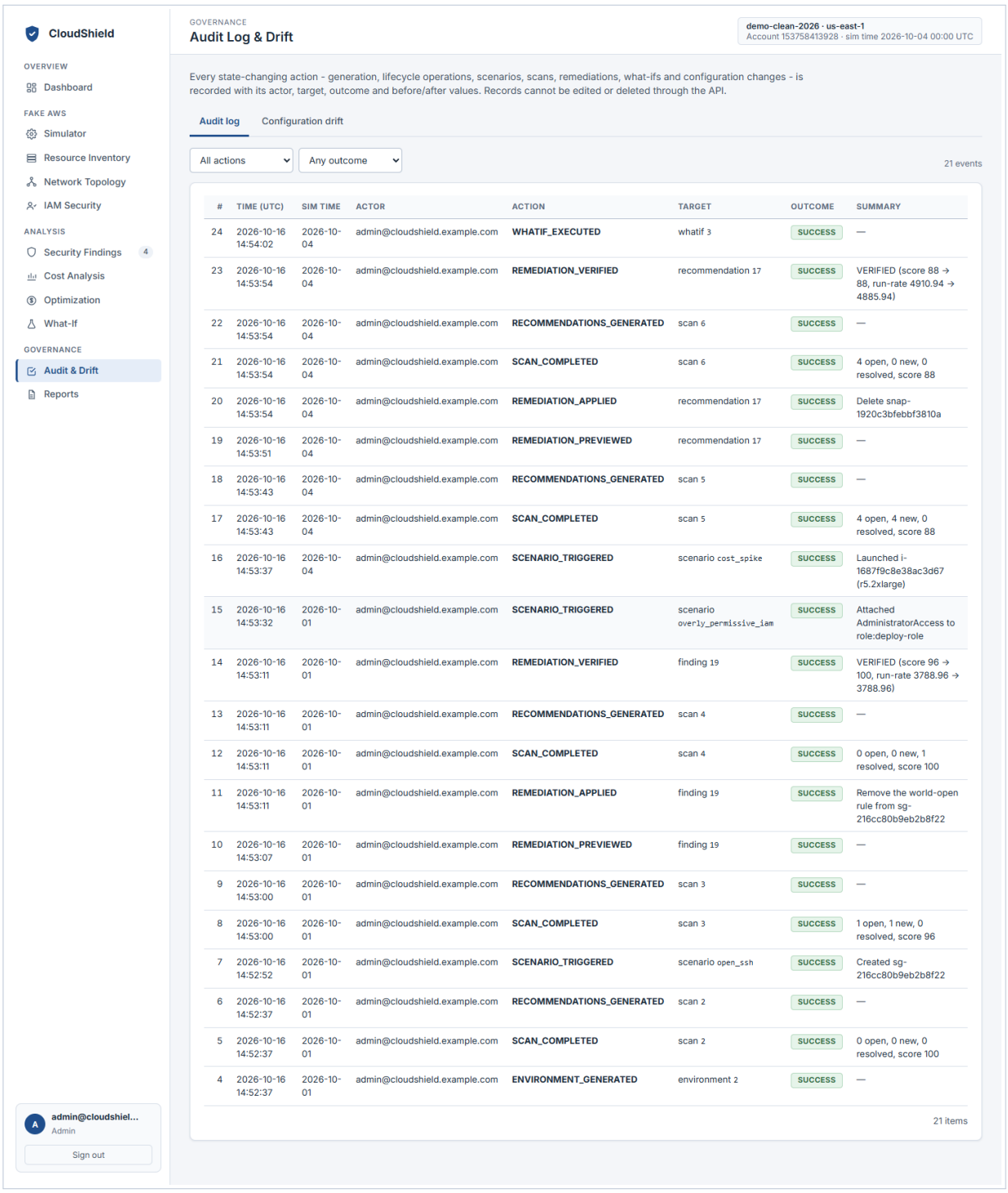


Figure 51. Audit log. Append-only record of generation, scenarios, scans, remediation, what-if and time changes with actor, outcome and details.

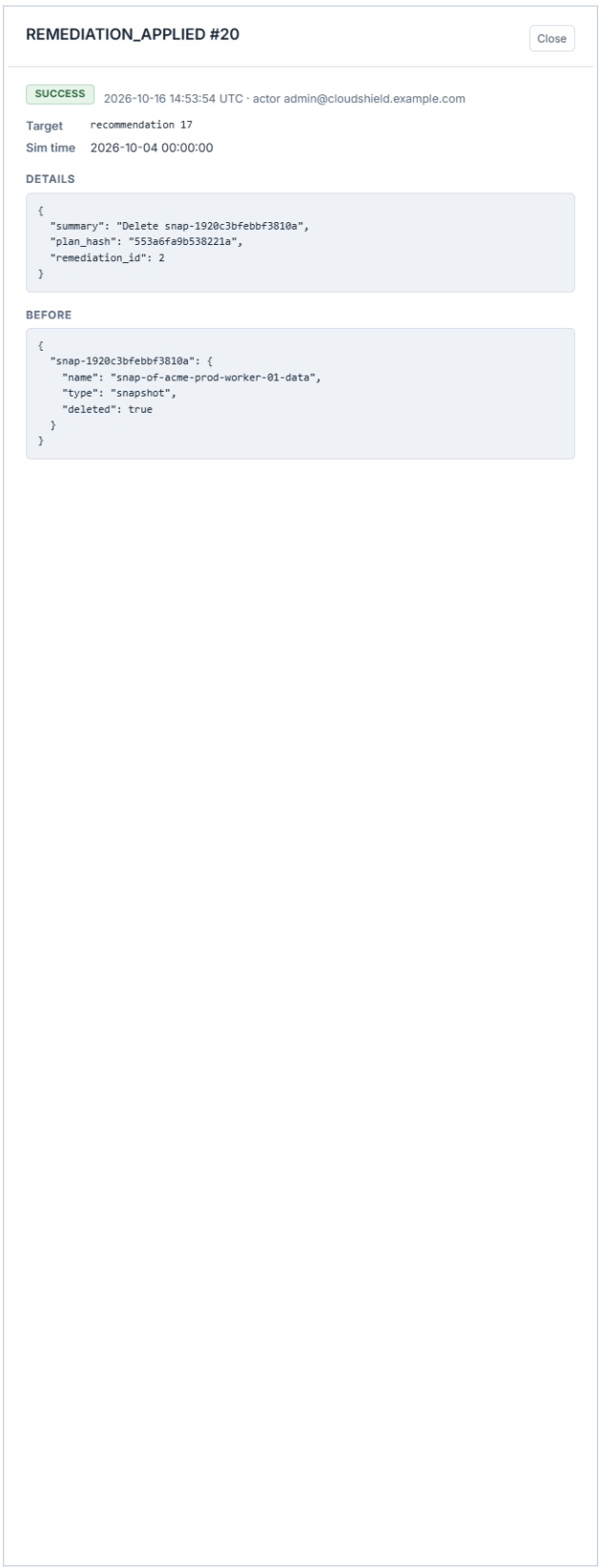


Figure 52. Audit event with before/after. A remediation event stores the changed fields before and after.

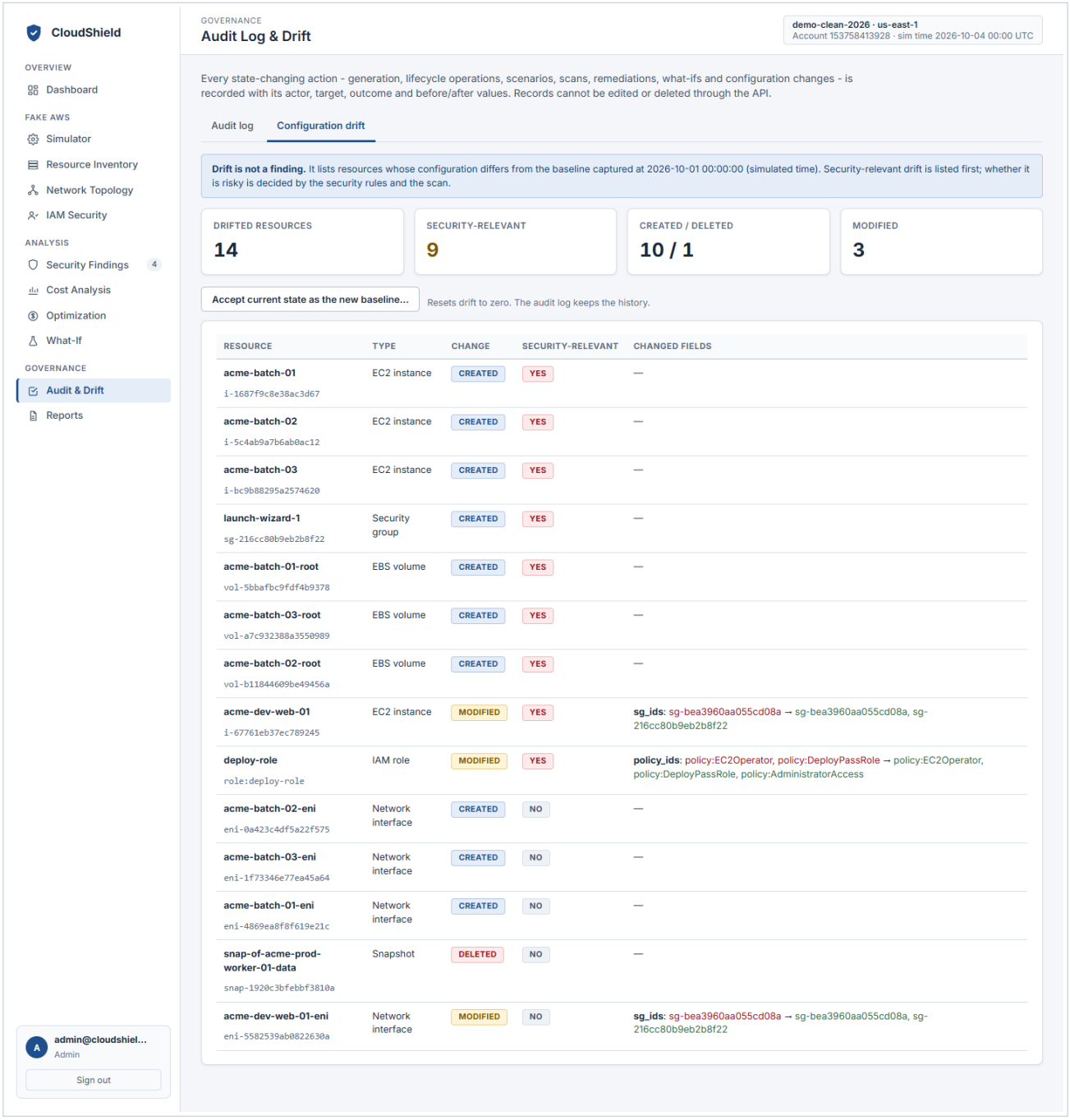


Figure 53. Configuration drift. Resources whose configuration differs from the saved baseline; security-relevant drift first. Drift is not a finding.

Reports and API documentation

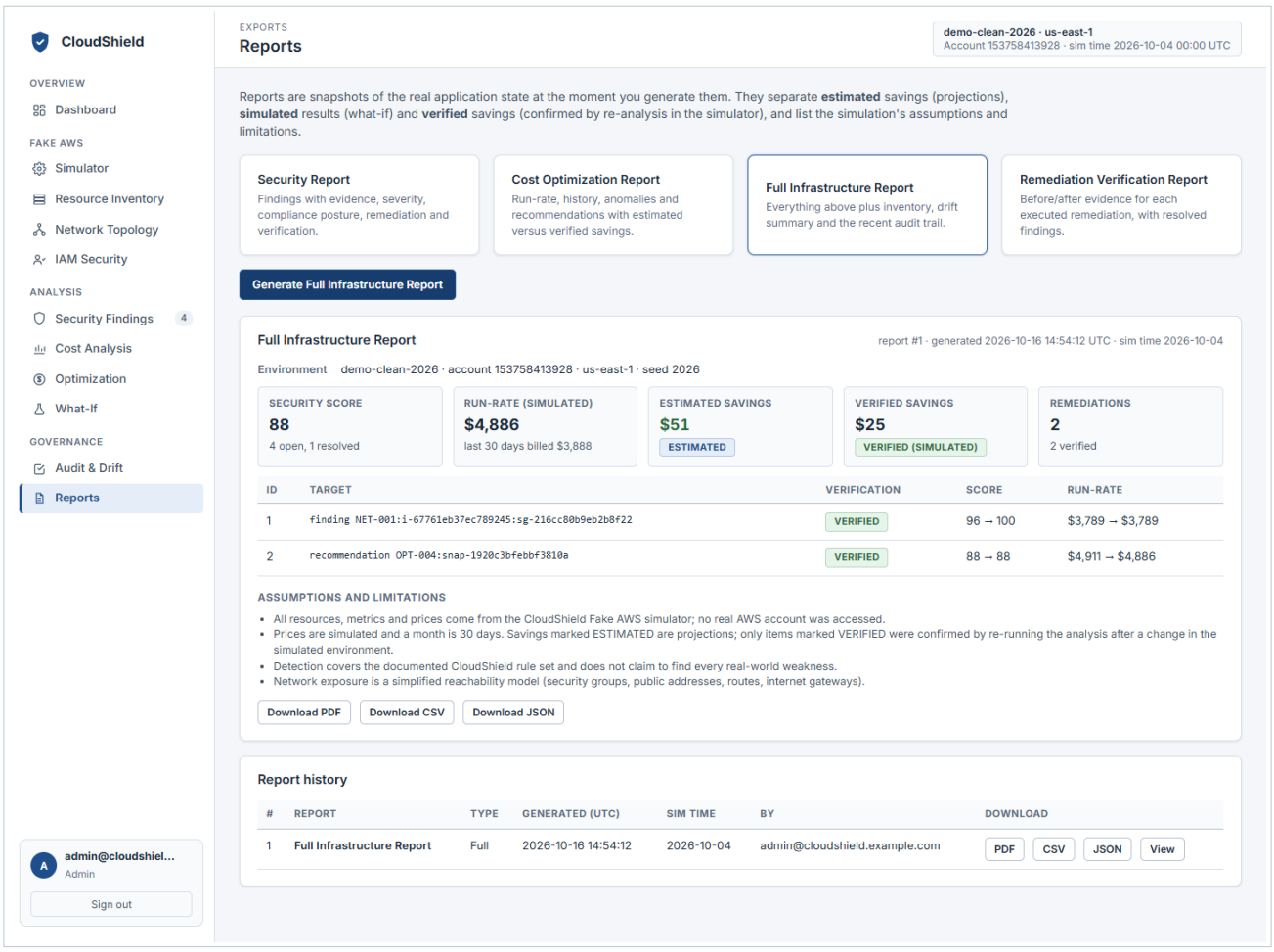


Figure 54. Generated report. Report built from persisted state: score, run-rate, estimated vs verified savings, remediation evidence, limitations.

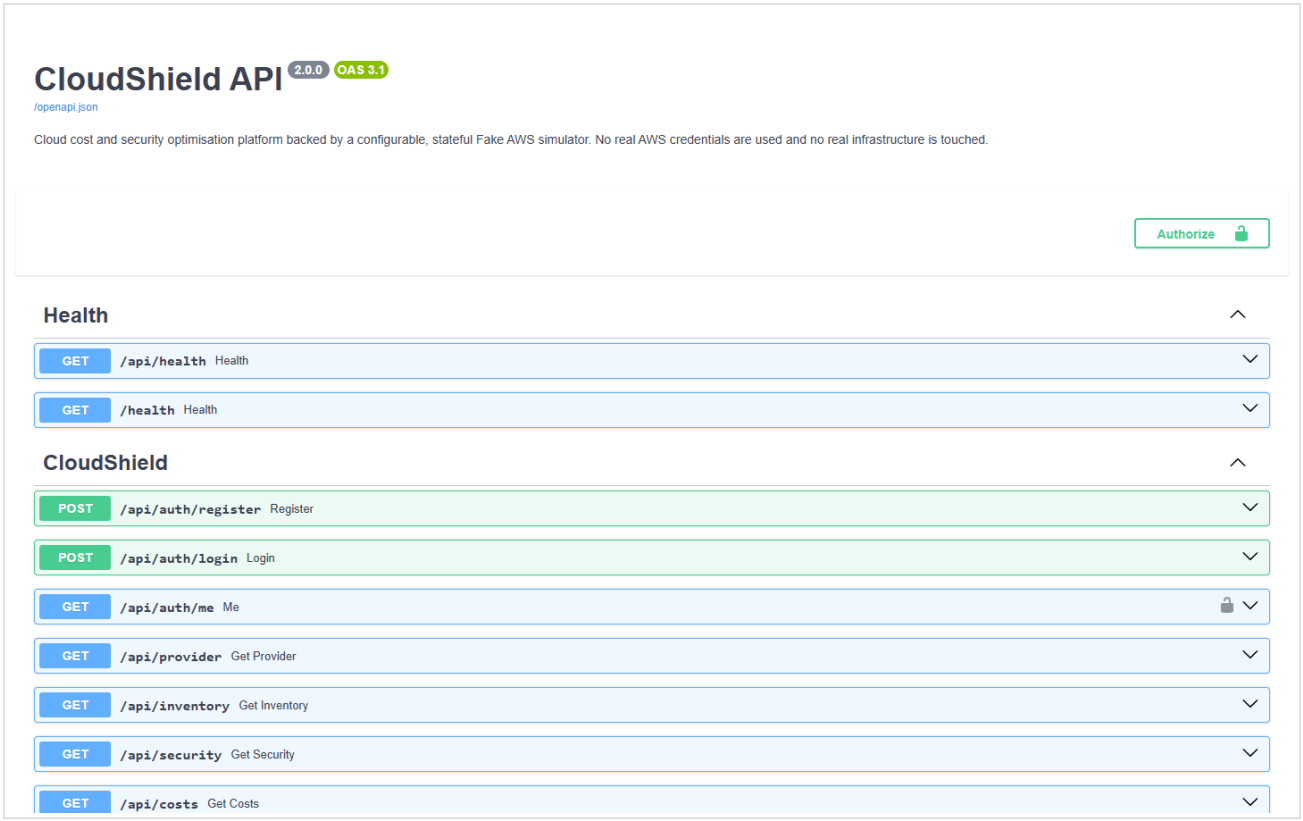


Figure 55. OpenAPI documentation. FastAPI /docs listing the simulator, analysis, remediation, what-if, audit and report endpoints.

8. Tools and technologies used

Area	Technology	Used for
Backend	Python 3.11, FastAPI 0.115, Uvicorn, Pydantic 2	REST API, validation, OpenAPI documentation
Persistence	PostgreSQL 16, SQLAlchemy 2, psycopg2	Users and all simulator state / history (JSONB columns)
Security	bcrypt (passlib), python-jose (JWT)	Password hashing, bearer tokens, role-based access
Simulator	Pure-Python domain model (dataclass-free dicts)	Deterministic generation, lifecycle, time, billing, scenarios
Frontend	Vanilla HTML / CSS / ES modules, inline SVG charts	11 views, topology and IAM graphs, no framework
Reports	ReportLab	Server-side PDF export; CSV and JSON exports
Testing	pytest 8, pytest-cov, httpx / Starlette TestClient	412 automated tests, 95% line coverage
Mutation testing	Cosmic Ray 8.4	Mutation score of rules, cost and validation code
UI automation	Playwright + Chrome	Real-browser walkthrough and screenshots
Containers	Docker Compose, nginx	Written; not executed on the development machine
Version control	Git, GitHub Flow	Feature branches merged into develop
Diagrams	Mermaid 10.9 (rendered offline in Chrome)	Architecture, DFD, ERD, UML, workflows
Local database	pgserver (embedded PostgreSQL 16.2)	Native PostgreSQL where Docker was unavailable

Table 2. Tools and technologies.

9. Test execution summary

Test file	Level	Tests	Passed	Failed	Time
test_api.py	API - auth, validation, purity of GET, audit, remediation, what-if, reports	35	35	0	46.9s
test_cost_time.py	Unit - billing, simulated time, anomalies, recommendations	41	41	0	5.5s
test_e2e_workflow.py	End-to-end - full workflow over HTTP	1	1	0	3.8s
test_frontend_contract.py	Frontend contract - UI to API wiring	66	66	0	0.2s
test_generator_config.py	Unit - configuration validation, deterministic generation, referential integrity	29	29	0	1.8s
test_hardening_config_billing.py		23	23	0	0.5s
test_hardening_engines.py		45	45	0	7.6s
test_integration_db.py	Integration - simulator and database	12	12	0	11.3s
test_lifecycle.py	Unit - validated lifecycle operations and prerequisites	39	39	0	3.2s
test_network_iam.py	Unit - network reachability, IAM evaluation, trust, escalation	26	26	0	2.5s
test_regression_legacy.py	Regression - original CloudShield behaviour	25	25	0	6.6s
test_security_rules.py	Unit - every detection rule, severity logic, scenarios	36	36	0	7.2s
test_whatif_drift.py	Unit - what-if isolation, remediation, drift, compliance, scenario revert	34	34	0	13.0s
Total	SQLite run	412	412	0	110.1s
Total	Same suite on PostgreSQL 16	412	412	0	85.5s

Table 3. Actual results from the JUnit reports (docs/testing/results/junit-*.xml). Line coverage of app/: 95%.

Full suite on SQLite with coverage 2026-10-16 21:26

```
$ python -m pytest tests -q -p no:cacheprovider --cov=app --cov-report=term --junitxml=../docs/testing/results/junit-sqlite.xml

.................................................................... [ 17%]
.................................................................... [ 34%]
.................................................................... [ 52%]
.................................................................... [ 69%]
.................................................................... [ 87%]
................................................                     [100%]
============================== warnings summary ==============================
..\.venv\Lib\site-packages\starlette\testclient.py:40
  C:\Users\bhuva\Downloads\abhinav_github\CloudShield-src\CloudShield\.venv\Lib\site-packages\starlette\testclient.py:40: DeprecationWarning: The
anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = typing.Callable[[], typing.ContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
- generated xml file: C:\Users\bhuva\Downloads\abhinav_github\CloudShield-src\CloudShield\docs\testing\results\junit-sqlite.xml -

---------- coverage: platform win32, python 3.11.9-final-0 ----------
Name                              Stmts   Miss  Cover
-----------------------------------------------------
app\auth.py                          46      2    96%
app\cost_engine.py                   49      4    92%
app\database.py                      13      0   100%
app\engines\__init__.py               0      0   100%
app\engines\analysis.py              12      0   100%
app\engines\compliance.py            23      0   100%
app\engines\cost.py                  94      0   100%
app\engines\drift.py                 41      1    98%
app\engines\iam.py                  149      0   100%
app\engines\network.py              194      5    97%
app\engines\optimization.py         144      3    98%
app\engines\remediation.py           34      0   100%
app\engines\security.py             197      0   100%
app\engines\whatif.py                39      0   100%
app\main.py                          99     33    67%
app\models.py                        72      0   100%
app\persistence.py                   36      3    92%
app\providers\__init__.py             8      0   100%
app\providers\base.py                30      7    77%
app\providers\mock_provider.py       19      0   100%
app\recommendation_engine.py         43      1    98%
app\routers\__init__.py               0      0   100%
app\routers\sim.py                  317      5    98%
app\routes.py                        71      0   100%
app\scan_service.py                  19      0   100%
app\schemas.py                      101      0   100%
app\security_engine.py               43      1    98%
app\services\__init__.py              0      0   100%
app\services\audit.py                28      0   100%
app\services\env_store.py            90      0   100%
app\services\inventory.py            41      0   100%
app\services\reporting.py           142      1    99%
app\services\scan.py                 68      2    97%
app\services\sim_service.py         293     21    93%
app\sim_models.py                   234      0   100%
app\simulator\__init__.py             0      0   100%
app\simulator\billing.py             36      0   100%
app\simulator\builders.py            77      6    92%
app\simulator\catalog.py             14      0   100%
app\simulator\config.py             191      0   100%
app\simulator\env.py                210      4    98%
app\simulator\generator.py          305      3    99%
app\simulator\lifecycle.py          675     81    88%
app\simulator\scenarios.py          173     19    89%
app\simulator\timeengine.py         156      7    96%
-----------------------------------------------------
TOTAL                              4626    209    95%

412 passed, 1 warning in 114.14s (0:01:54)
```

exit code **0** · 119.6s · working directory backend

Figure 56. Full suite on SQLite with coverage. Complete test run with line coverage of the application package.

Unit tests: generator, lifecycle, network, IAM, rules, cost, time, what-if, boundary hardening 2026-10-16 21:23

```
$ python -m pytest tests/test_generator_config.py tests/test_lifecycle.py tests/test_network_iam.py tests/test_security_rules.py tests/test_cost_time.py tests/te
st_whatif_drift.py tests/test_hardening_config_billing.py tests/test_hardening_engines.py -q -p no:cacheprovider

.................................................................... [ 26%]
.................................................................... [ 52%]
.................................................................... [ 79%]
.....................................................                [100%]
============================== warnings summary ==============================
..\.venv\Lib\site-packages\starlette\testclient.py:40
  C:\Users\bhuva\Downloads\abhinav_github\CloudShield-src\CloudShield\.venv\Lib\site-packages\starlette\testclient.py:40: DeprecationWarning: The
anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = typing.Callable[[], typing.ContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
273 passed, 1 warning in 14.06s
```

exit code **0** · 17.8s · working directory backend

Figure 57. Unit tests: generator, lifecycle, network, IAM, rules, cost, time, what-if, boundary hardening. Unit-test evidence: seed reproducibility, lifecycle validation, reachability, IAM evaluation, detection rules, cost arithmetic, anomaly detection, what-if isolation.

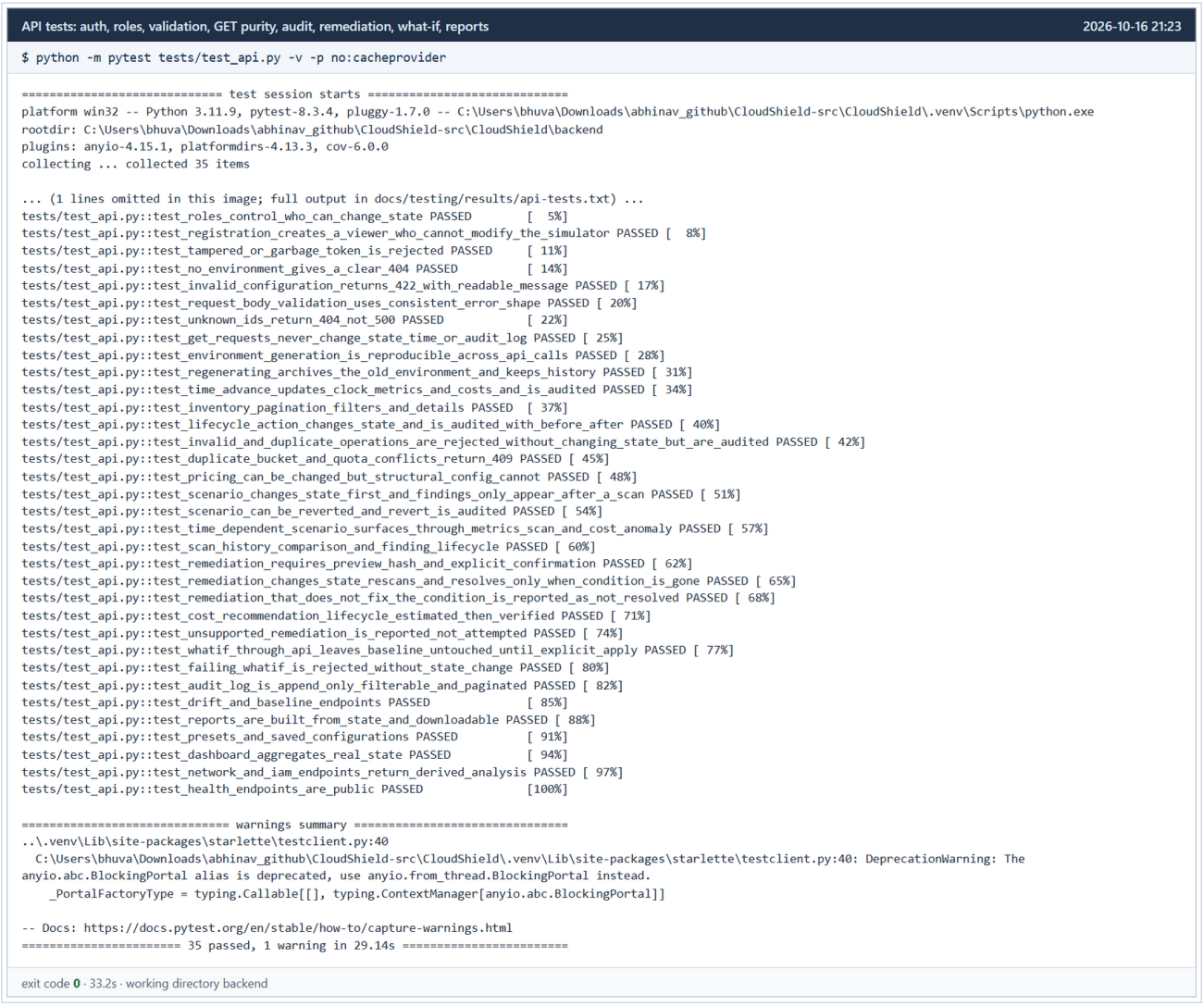


Figure 58. API tests: auth, roles, validation, GET purity, audit, remediation, what-if, reports. API-level evidence: valid and invalid input, unauthorised calls, missing resources, duplicate operations, read-only GETs, what-if isolation, audit creation.

Defects found and fixed while testing

- **IAM trust evaluation**: a role trusting the account root returned early and hid a later wildcard or explicit principal; fixed to prefer the most permissive entry.
- **Snapshot billing**: steady-state history showed an artificial ramp because policy snapshots started with one copy; now the full retention window is generated and policy snapshots are billed incrementally.
- **Recommendation after resize**: metrics recorded before a resize described the old size and could trigger a second recommendation; the rules now ignore pre-resize metrics.
- **Evidence overwrite**: a finding's evidence lost the rule object because a text summary shared the key; renamed.
- **Live configuration patch**: unknown fields were silently ignored and nested price changes replaced the whole price table; now rejected (422) or deep-merged.
- **Topology highlighting**: ordinary HTTP/HTTPS world-open rules were flagged as exposure; only sensitive-port or all-port rules are flagged now.

10. Conclusion

The CloudShield prototype was turned into a working, tested platform: a seeded Fake AWS simulator with persistent state, 20 state-derived detection rules, a cost model with anomaly detection and recommendations, a preview-confirm-apply-verify remediation workflow, isolated what-if analysis, an append-only audit trail with drift detection, role-based access and downloadable reports. 412 automated tests pass on SQLite and 412 of 412 on

PostgreSQL 16 with 95% line coverage; the real UI walkthrough completes without browser errors. Limits that matter for an evaluator: Docker images were written but not built here, mutation testing used a seeded sample of mutants, and all prices, metrics and 'AWS' resources are simulated.

Appendix A. Evidence register

File (docs/screenshots/)	Shows	Demonstrates
application/01-login.png	Sign-in screen	Login view; credentials are validated by the API, not hard-coded in the page.
application/02-login-rejected.png	Rejected sign-in	A wrong password is refused with a generic message (HTTP 401).
application/03-dashboard.png	Dashboard (default environment)	KPIs, security posture, cost trend, cost by service and compliance derived from the persisted simulated environment.
application/04-dashboard-after-scenarios.png	Dashboard after scenarios and a rescan	Dashboard reflects the new findings, the cost anomaly and updated compliance posture.
application/05-report-generated.png	Generated report	Report built from persisted state: score, run-rate, estimated vs verified savings, remediation evidence, limitations.
application/06-api-docs.png	OpenAPI documentation	FastAPI /docs listing the simulator, analysis, remediation, what-if, audit and report endpoints.
application/07-viewer-readonly.png	Read-only viewer role	A viewer sees a notice and cannot generate environments or change state (also enforced by the API with HTTP 403).
cost/01-cost-anomaly.png	Cost analysis with a detected anomaly	Daily cost chart with the spike flagged (z-score above baseline), by-service breakdown and contributors.
cost/02-cost-by-service.png	Daily cost by service	Stacked daily cost per service from simulated billing.
cost/03-optimization.png	Optimization recommendations	Each recommendation shows evidence, estimated saving (labelled ESTIMATED), risk and the action to take.
cost/04-recommendation-evidence.png	Recommendation evidence and assumptions	Supporting metrics, assumptions, risk and the executable plan.
cost/05-recommendation-whatif.png	Recommendation what-if preview	Cost and security effect simulated before applying.
cost/06-recommendation-verified.png	Recommendation applied and verified	Run-rate before/after shows the verified (simulated) saving.
cost/07-recommendation-status-verified.png	Verified recommendation	Estimated and verified savings are shown separately.
docker/compose-static-validation.png	Static validation of docker-compose.yml and Dockerfiles	Compose file parses, defines db/backend/frontend with health checks, a named volume and dependency conditions (not a substitute for a real build).
docker/docker-availability.png	Docker availability check on the development machine	Honest record: Docker is not installed here, so image build / compose up could not be executed.
docker/native-startup-persistence.png	Clean start, health check and persistence across restart (native, PostgreSQL 16)	The same stack (FastAPI + PostgreSQL) started from an empty database, health-checked, restarted, with identical data afterwards.
security/01-findings-clean.png	Findings on the clean environment	Before any scenario: no critical or high findings are detected.
security/02-sg-before-remediation.png	Security group BEFORE remediation	The scenario's security group allows tcp/22 from 0.0.0.0/0 (resource state, before the fix).
security/03-findings-after-scan.png	Findings after the scenario and a scan	A new HIGH finding NET-001 appears, derived from the changed security group, public address and route.
security/04-finding-evidence.png	Finding with evidence	Explanation, impact, step-by-step reachability proof, detection rule, severity reason and timeline.
security/05-remediation-preview.png	Remediation preview (PROPOSED)	Plan, risk, and a what-if comparison computed on an isolated copy: score before/after, cost, availability, config diff. Nothing has changed yet.
security/06-remediation-verified.png	Remediation executed and verified	The change was applied to the simulated state, the analysis re-ran, and the condition is no longer detected (VERIFIED).
security/07-sg-after-remediation.png	Security group AFTER remediation	Same resource after remediation: the world-open SSH rule is gone (compare with the BEFORE screenshot).

File (docs/screenshots/)	Shows	Demonstrates
security/08-findings-resolved.png	Resolved finding kept in history	The finding is RESOLVED because a re-scan no longer detects it; the record and its history remain.
security/09-finding-resolution-detail.png	Resolved finding detail	Timeline, resolving scan, remediation record and verification result.
security/10-remediation-history.png	Remediation and verification history	Proposed -> executed -> verified, with score and run-rate before/after.
security/11-scan-comparison.png	Scan history and comparison	Compare two scans: resolved/introduced findings and score change.
security/12-detection-rules.png	Detection rules catalogue	Each rule's logic and severity basis is documented and visible.
security/13-iam-escalation.png	IAM privilege escalation after a permissive role	AdministratorAccess on a workload role creates escalation paths from ordinary users; red edges show the chain.
simulator/01-configuration.png	Simulator configuration	Seed, region, resource counts, utilization, misconfiguration rates, quotas, time, pricing and scenario probabilities.
simulator/02-configuration-security-rates.png	Misconfiguration rates and scenario probabilities	Clean preset: misconfiguration rates are zero, so no flaw exists until a scenario creates it.
simulator/03-generate-confirm.png	Generate confirmation	Generation archives (does not delete) the previous environment and its history.
simulator/04-environment-generated.png	Environment generated	Active environment panel shows the seed, simulated clock, fingerprint and resulting resource counts (16 EC2 instances).
simulator/05-environments-history.png	Environment history	The earlier environment is ARCHIVED, not erased; scans and audit history remain.
simulator/06-inventory.png	Resource inventory	Paginated inventory of every simulated resource with state, key attributes, environment and monthly cost.
simulator/07-resource-detail.png	Resource detail drawer	Per-resource properties, tags, metrics, relationships, network reachability explanation and the validated operations available.
simulator/08-network-topology-clean.png	Network topology (no exposure)	VPCs, public/private subnets, internet gateways, instances, databases and security groups. Nothing is exposed on a sensitive port.
simulator/09-iam-analysis.png	IAM identity analysis	Principals, admin detection, MFA and access-key status, trust relationships and the escalation graph.
simulator/10-time-and-scenarios.png	Simulated time and scenarios	Manual clock controls and the ten scenarios; each changes state and names the rules expected to detect it.
simulator/11-scenario-triggered.png	Scenario applied	State changed (a world-open SSH rule was added); the 'state changed since last scan' chip appears because no finding exists until a scan runs.
simulator/12-topology-exposed.png	Network topology with an exposed instance	The instance is outlined red with 'EXPOSED 22' because the network model finds SSH reachable from the internet.
simulator/13-whatif-result.png	What-if comparison	Baseline vs hypothetical: cost, security score, availability, compliance and config diff; the baseline fingerprint is verified unchanged.
simulator/14-audit-log.png	Audit log	Append-only record of generation, scenarios, scans, remediation, what-if and time changes with actor, outcome and details.
simulator/15-audit-event-detail.png	Audit event with before/after	A remediation event stores the changed fields before and after.
simulator/16-configuration-drift.png	Configuration drift	Resources whose configuration differs from the saved baseline; security-relevant drift first. Drift is not a finding.
testing/api-tests.png	API tests: auth, roles, validation, GET purity, audit, remediation, what-if, reports	API-level evidence: valid and invalid input, unauthorised calls, missing resources, duplicate operations, read-only GETs, what-if isolation, audit creation.
testing/frontend-contract-tests.png	Frontend integration contract tests	Every API call made by the UI exists in the backend OpenAPI document; every button/handler is wired.
testing/full-suite-coverage.png	Full suite on SQLite with coverage	Complete test run with line coverage of the application package.
testing/full-suite-postgres.png	Full suite on PostgreSQL 16	The same suite executed against a real PostgreSQL 16 server (TEST_DATABASE_URL).

File (docs/screenshots/)	Shows	Demonstrates
testing/integration-tests.png	Integration tests: simulator <-> database, engines <-> persisted history	Integration-testing evidence (simulator and database, optimistic locking, transactions, scan reconciliation).
testing/mutation-summary.png	Mutation testing summary (Cosmic Ray, two rounds)	Mutation-testing evidence: per-module killed/survived counts, round 1 (targeted tests) and final (after hardening tests).
testing/regression-tests.png	Regression tests: original CloudShield behaviour	Regression evidence: original authentication, mock provider, label-based engines, original API routes and persistence still behave as before.
testing/unit-tests.png	Unit tests: generator, lifecycle, network, IAM, rules, cost, time, what-if, boundary hardening	Unit-test evidence: seed reproducibility, lifecycle validation, reachability, IAM evaluation, detection rules, cost arithmetic, anomaly detection, what-if isolation.
version-management/git-branches.png	Git branches	Branches used by the GitHub-Flow workflow (main, develop, feature branches).
version-management/git-graph.png	Git history graph	Meaningful commits and --no-ff merges of feature branches into develop.
version-management/git-status-remote.png	Repository, remote and working-tree state	Remote URL and clean working tree after the commits (nothing is pushed by the tooling).

Table 4. Every screenshot used in this report with its purpose.